

# Tachyon Guide

The Tachyon scaling architecture, at design stage: what is specified, what is designed,  
what is open

m@rek.onl

## Abstract

This volume of *The Zcash Arboretum* documents a protocol that does not yet exist. Tachyon is the proposed rearchitecture of Zcash payments around proof-carrying wallet state: the wallet folds each block's effects into a recursive proof, so synchronisation ceases to scale with chain length, and the chain ceases to carry the wallet's burdens. Because the design is in motion, this volume's organising device is a strict classification: every claim is *specified*, *designed but unspecified*, or an *open problem*. The strongest chapter is the last kind. The reader should expect this volume to age faster than its siblings and to be revised as the design lands.

# Contents

|          |                                                                                  |           |
|----------|----------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Scope, sources, and method</b>                                                | <b>3</b>  |
| 1.1      | Why a design-stage volume . . . . .                                              | 3         |
| 1.2      | The three-bin method . . . . .                                                   | 3         |
| 1.3      | The source landscape . . . . .                                                   | 3         |
| 1.4      | Standing . . . . .                                                               | 5         |
| 1.5      | What this volume can and cannot deliver . . . . .                                | 5         |
| <b>2</b> | <b>The proof-carrying wallet</b>                                                 | <b>5</b>  |
| 2.1      | Wallet state as an accumulation instance . . . . .                               | 6         |
| 2.2      | What the fold removes . . . . .                                                  | 7         |
| 2.3      | Validator-side consequences . . . . .                                            | 8         |
| 2.4      | Payment delivery . . . . .                                                       | 8         |
| <b>3</b> | <b>The design as published</b>                                                   | <b>9</b>  |
| 3.1      | Bundles and tachygrams . . . . .                                                 | 10        |
| 3.2      | The key hierarchy . . . . .                                                      | 11        |
| 3.3      | Notes and commitments . . . . .                                                  | 12        |
| 3.4      | Epochs and the GGM nullifier tree . . . . .                                      | 12        |
| 3.5      | Anchors, stamps, and the proof tree . . . . .                                    | 13        |
| 3.6      | Aggregation . . . . .                                                            | 15        |
| 3.7      | Authorization . . . . .                                                          | 16        |
| 3.8      | Transaction identifiers and domain separators . . . . .                          | 17        |
| <b>4</b> | <b>What is specified today</b>                                                   | <b>19</b> |
| 4.1      | ZIP 321 as substrate, and the four groundwork drafts . . . . .                   | 19        |
| 4.2      | Ragu at commit 830bbcd: the proof layer as code . . . . .                        | 20        |
| 4.3      | The <code>zcash_tachyon</code> crate: integration against a mock . . . . .       | 22        |
| <b>5</b> | <b>Designed but unspecified, and open problems</b>                               | <b>22</b> |
| 5.1      | Designed but unspecified . . . . .                                               | 22        |
| 5.1.1    | Nullifier evolution and re-randomization for oblivious synchronisation . . . . . | 22        |
| 5.1.2    | The PIR detection design . . . . .                                               | 23        |
| 5.1.3    | The sync-service trust model . . . . .                                           | 24        |
| 5.1.4    | Epoch parameterization . . . . .                                                 | 24        |
| 5.1.5    | The rollover soundness model . . . . .                                           | 25        |
| 5.2      | Open problems . . . . .                                                          | 26        |
| 5.2.1    | Timing-correlation linkage . . . . .                                             | 26        |
| 5.2.2    | Data availability, liveness, and incentives . . . . .                            | 26        |
| 5.2.3    | Recoverability . . . . .                                                         | 27        |
| 5.2.4    | Tip-nullifier exposure versus the obliviousness claim . . . . .                  | 27        |
| 5.3      | Quantitative claims without a published basis . . . . .                          | 28        |
| 5.4      | Revisit triggers . . . . .                                                       | 28        |
| <b>6</b> | <b>Security posture and outlook</b>                                              | <b>29</b> |
| 6.1      | The conservation law . . . . .                                                   | 29        |
| 6.2      | The post-quantum intersection . . . . .                                          | 30        |
| 6.3      | Relation to Crosslink and the upgrade environment . . . . .                      | 31        |
| 6.4      | The checkability boundary . . . . .                                              | 31        |

# 1 Scope, sources, and method

Project Tachyon is a proposed redesign of Zcash’s shielded protocol around three ideas: wallet state becomes proof-carrying data, so a wallet proves the validity and unspentness of its own notes and a verifier checks a proof rather than replaying history; nullifiers *evolve* per epoch, so consensus checks duplicates over a bounded window and permanently prunes older state; and untrusted *oblivious synchronization services* keep wallets’ proofs current without learning their transaction details (Bowe, “Tachyon: Scaling Zcash with Oblivious Synchronization”, 2025-04-02; Bowe–Miers, eprint 2025/2031, 2025-11-03). The intended proof layer is Ragu, a recursive-proof (PCD) framework over the Pasta cycle with no trusted setup (`tachyon-zcash/ragu @ 830bbcda`). None of this is deployed; none of it is specified at consensus rigor.

## 1.1 Why a design-stage volume

The sibling volumes document a deployed protocol: every claim in the *Ironwood Guide* cites shipped code and a final specification. Tachyon has neither — no protocol specification, no merged ZIP, no code on any network. It nevertheless warrants a volume now, for two reasons. First, its standing: of all proposals in the February 2026 NU7 sentiment poll, Tachyon drew the most unambiguous support (§1.4), so it is the direction Zcash’s shielded layer is most likely to take, and the design deserves scrutiny *before* it hardens into consensus rules. Second, its volatility: the design is single-committer, actively churning, and self-admittedly stale in places (§1.3); fixing its current state against commit-pinned sources creates the baseline any later normative work can be diffed against. The cost of writing now is stated plainly: this volume can assert nothing normatively, and its strongest chapter is the one on open problems.

## 1.2 The three-bin method

Design-stage material invites a failure mode the deployed-protocol volumes never face: prose that reads as specification but rests on a blog post. We guard against it by classifying every claim in this volume into exactly one of three bins, and keeping the bin visible in the prose.

**Definition 1.1** (Three-bin classification). A claim about Tachyon is exactly one of:

- (i) *specified* — a normative artifact pins it down (a merged ZIP, the protocol specification, or deployed code); we cite the artifact, with file and commit;
- (ii) *designed but unspecified* — an informal source (design book, eprint note, blog post, forum post, issue) describes a concrete mechanism, but no specification fixes it; we cite the source and its date (plus commit for repositories) and state what a specification would still need to pin down;
- (iii) *an open problem* — no source, formal or informal, resolves it; where the designers themselves acknowledge the gap, we quote them.

The bins are this volume’s organizing device. Each subsequent section groups its material by bin — §4 carries the specified bin exhaustively, §5.1 and §5.2 the lower two — or closes each construction with a *Classification* paragraph naming its bin; a claim appearing outside such a block names its bin inline. Claims about deployed or extant code cite the crate and file; claims about design carry a date, because the design has already pivoted once (§1.3) and undated statements from the wrong era are actively misleading.

## 1.3 The source landscape

Table 1 lists the primary sources, each pinned to a commit or date. All repository facts below were verified firsthand at the stated commits; web sources were fetched 2026-07-08.

| Artifact                            | Pin / date               | Rigor                       |
|-------------------------------------|--------------------------|-----------------------------|
| eprint 2025/2031 (Bowe–Miers)       | rev. 2025-11-03          | 8-page note; no theorems    |
| tachyon-zcash/tachyon (book, crate) | 26fdc165, 2026-07-07     | design book; crypto stubbed |
| tachyon-zcash/ragu                  | 830bbcd, 2026-07-05      | pre-release code; unaudited |
| tachyon-zcash/zips PRs #1–#4        | opened 2026-06-02; open  | drafts, not upstreamed      |
| zcash/zips (upstream)               | c6b70358, 2026-07-05     | zero Tachyon ZIPs           |
| Blog, forum t/50789, ZK Podcast 388 | 2025-04-02 to 2026-01-31 | designer prose              |

Table 1: Primary Tachyon sources, pinned. Verification date 2026-07-08.

**The eprint note.** The canonical publication is Bowe and Miers, “A Note on Notes: Towards Scalable Anonymous Payments via Evolving Nullifiers and Oblivious Synchronization” (eprint 2025/2031; received 2025-11-02, revised 2025-11-03). It is a self-described “short note” of eight pages: it fixes the two central ideas and their motivation, states the fatal risks in the authors’ own words, and contains zero theorems and no formal security definitions. It anchors bin (ii) for the core concepts and, unusually, much of bin (iii): its frankest passages are the acknowledged unsolved problems.

**The project book and crate.** The repository tachyon-zcash/tachyon @ 26fdc165 (2026-07-07) carries the most concrete design material: an mdBook elaborating nullifier derivation, the proof tree, the anchor chain, aggregation, and a ZIP dependency map, beside the zcash\_tachyon crate. Three caveats govern every citation of it. First, it is a single-committer artifact (git shortlog lists only Tal Derei) with no independent publication of its post-eprint machinery. Second, the crate does not implement the load-bearing cryptography: its README lists note commitments, nullifier derivation, and proof creation, merging, and verification as “Stubbed (pending Ragu PCD and Poseidon integration)” (README.md line 49 @ 26fdc165). Third, the book is self-admittedly stale: issue #121 (2026-05-26) records a pivot in the payment design — “Since we’re using PIR to make repeated payments flows without OOB, there are stale parts of the book that will need to change. We’re retaining in-band secret distribution for recover from seed flows” — so every statement from the out-of-band era (the 2025 blog posts, the book’s key chapter) must be dated pre-2026-05-26, and we date them throughout.

**The ZIP landscape.** The project’s ZIP program exists at two rigors, and the split is the clearest single measure of Tachyon’s maturity. The five *core* ZIPs — Shielded Protocol, Bundle, Accumulator, Aggregator, Oblivious Synchronization (tracking issues #103–#107) — are empty stubs: each book file carries only a tracking link and blank section headings (book/src/zips/ @ 26fdc165, 216–245 bytes per stub). Four *peripheral* amendments — to ZIP 317 (fees), ZIP 209 (pool balance), ZIP 221 (history tree), and ZIP 248 (bundle registration) — exist as ZIP-0-conformant drafts, but only as PRs #1–#4 on the project’s own zcash/zips fork, open since 2026-06-02 and not upstreamed; the ZIP 248 they amend is itself unmerged upstream (open PR zcash/zips#1156, idle since 2026-05-26). Upstream zcash/zips @ c6b70358 contains zero Tachyon ZIPs: a repository-wide search finds only the ZIP editor list, one author affiliation in protocol/protocol.tex, and one footnote citing the founding blog post. The NU7 deployment draft (draft-arya-deploy-nu7.md, Status Draft, branch 0x77190AD8, activation heights TBD) schedules no Tachyon ZIP — nor any other.

**Ragu.** The proof layer lives in tachyon-zcash/ragu @ 830bbcd (2026-07-05): a working PCD pipeline over the Pasta cycle, at workspace version 0.0.0, with no release, no crates.io publication, no audit, and in-code security placeholders (the Fiat–Shamir domain tag is literally b"FIXME"; crates/ragu\_pcd/src/lib.rs lines 50–51). We treat Ragu’s code state in its own section; here it matters only as a pinned source.

**Informal sources.** The remaining corpus is designer prose: Bowe’s blog (founding post 2025-04-02; “Tachyaction at a Distance” 2025-05-15; post-quantum post 2025-10-16, after which the index ends and the promised deep-dive posts never appeared), the forum megathread t/50789 (28 posts, 2025-04-02 to 2026-01-31), the project site (dateless roadmap), and the ZK Podcast 388 transcript (2026-01-21). These sources carry bin (ii) intent and bin (iii) admissions, never specification. We quote them with dates. The “two orders of magnitude” headline is the project’s own (tachyon.z.cash) with no published derivation; the circulating TPS estimates are analyst characterization, not project claims; no primary document derives either (§5.3).

## 1.4 Standing

Two institutional facts frame the volume. First, governance: in the Zcash Foundation’s February 2026 NU7 sentiment poll, Tachyon drew “universal or near-universal support across every group” — “the most unambiguous signal from the entire poll” — with the stated next step to “assess technical readiness and scope for inclusion in NU7” (zfnd.org, 2026-02-23). Support is not scheduling: as noted above, the NU7 deployment draft lists no ZIPs at all. Second, editorship: Sean Bowe and Tal Derei serve as ZIP editors “associated with Project Tachyon” (`zip-0000.rst` line 196 @ c6b70358), and the protocol specification’s author block carries the same affiliation (`protocol/protocol.tex` line 2953). The people who would shepherd Tachyon ZIPs through the process are the protocol’s designers — an institutional fact we record without inference.

## 1.5 What this volume can and cannot deliver

In bin (i) we can place only the standing environment: the poll outcome and editorships above, the adjacent network-upgrade context (NU6.2 final, NU6.3 in draft), the out-of-band substrate ZIPs the design leans on, and Ragu’s code state at 830bbcd. Bin (ii) holds the entire core protocol — evolving nullifiers, dual-nullifier spends, the anchor chain, the proof tree, delegation, aggregation, the distinct shielded pool the notes inhabit — every part of it cited to the book at 26fdc165 as non-normative, dated, and in places already superseded. No byte format, consensus rule, security theorem, or performance figure appears in this volume as a project claim, because none exists at that rigor.

The strongest chapter is therefore the one on open problems, and this is not a concession but the point: the best-documented statements in the corpus are the designers’ own acknowledgments of what remains unsolved — timing correlation by synchronization services (“fatal to the entire approach”; eprint 2025/2031 §1.3), data availability after validator pruning (“unspendable, potentially permanently”; eprint 2025/2031 §1.4), and recoverability, whose entire current written resolution is one sentence in issue #121. The events that would obsolete parts of this volume are catalogued as revisit triggers in §5.4; we revisit at any of them.

## 2 The proof-carrying wallet

An Orchard wallet reconstructs its state from the public chain. To detect its notes it trial-decrypts every published Action from its birthday height onward, and to keep its authentication paths current it ingests every published commitment — its own and everyone else’s — into a pruned local tree (the *Ironwood Guide*, §“Transmission and detection of a note: encryption, trial decryption, and synchronisation” and §“Proof of ownership of *some* valid note: the commitment tree”; the deployed mechanism is librustzcash’s *shardtree*). Both costs scale with global chain traffic, not with the wallet’s own activity. Tachyon inverts the relationship: the wallet holds a *proof* of its own spendability — “this note entered the pool, and no nullifier of it has been published since” — and advances that proof by folding in each block’s deltas, retaining nothing else about the chain. The concept is Bowe’s founding post (“Tachyon: Scaling Zcash with Oblivious Synchronization”, 2025-04-02); the publication of record is the Bowe–Miers note (eprint 2025/2031, 2025-11-02), eight

pages with no theorems; the concrete elaboration is the project book (tachyon-zcash/tachyon, commit 26fdc165, 2026-07-07).

Nothing in this section is consensus-specified. Upstream zcash/zips carries no Tachyon ZIP (repository state at commit c6b70358, 2026-07-05), and the core “Tachyon Shielded Protocol” ZIP is an empty stub (tachyon-zcash/tachyon issue #103). We therefore tag every claim with its bin: *specified* (a citable artifact at spec or code rigor), *designed but unspecified* (an informal source, together with what a specification would still need to pin down), or *open problem*.

## 2.1 Wallet state as an accumulation instance

The state object is the *spendable*, the triple

$$(\text{cm}, \text{nf}_e, \text{anchor}),$$

carried as a proof header: the note’s commitment, its nullifier for the current epoch  $e$ , and a running commitment to the pool state up to which absence has been proven (tachyon book, `src/proof-tree.md` and `src/nullifiers.md` at 26fdc165). A Tachyon note has a distinct nullifier per epoch: from the note’s trapdoor  $\psi$  and the wallet’s nullifier key  $\text{nk}$  a per-note master key  $\text{mk} := \text{Poseidon}_{\text{Tachyon-NfPrefix}}(\psi, \text{nk})$  roots a GGM tree whose leaf at epoch  $e$  yields  $\text{nf}_e$ , and the commitment  $\text{cm} := \text{Poseidon}_{\text{Tachyon-CmDerive}}(\text{rcm}, \text{pk}, v, \psi)$  digests  $\psi$  and  $\text{pk}$  (which pins  $\text{nk}$ ), so a note’s entire nullifier sequence is frozen when its commitment enters the pool. Construction 3.4 states the derivation in full — the climb and leaf domains, and the working depth and arity the crate pins (§3.4) while the book leaves them symbolic. The proof behind the header certifies a single sentence: the note whose commitment is  $\text{cm}$  was minted — its creation *stamp*, the proof object recording its transaction’s published commitments and nullifiers (Definition 3.1), is folded into the pool state — and none of its nullifiers, from the creation epoch through  $e$ , appears in the pool up to anchor.

The fold advances this state without the wallet touching the chain. A synchronisation service — holding only an opaque window of the note’s future nullifiers, never the note,  $\text{cm}$ ,  $\psi$ , or  $\text{mk}$  — assembles an *Unspent* segment: **UnspentSeed** proves one wallet-supplied nullifier absent from one stamp’s tachygram set (a *tachygram* is a 32-byte field element representing a note commitment or a nullifier, the two indistinguishable to consensus), **EmptyBlockUnspentSeed** covers empty blocks, **UnspentFuse** composes contiguous segments, and **UnspentEpochFuse** crosses an epoch boundary, splicing the completed epoch’s nullifier into the segment’s elapsed history. The wallet then binds and folds: **VerifyUnspent** proves the segment’s crossed nullifiers and its tip are genuine GGM leaves of the note identified by  $\text{cm}$ , and **SpendableLift** consumes the prior spendable together with the verified segment, checks commitment equality, nullifier continuity, and anchor adjacency, and emits the advanced spendable (book, `src/proof-tree.md` at 26fdc165). Each step consumes at most two prior proofs and emits one — proof-carrying data over the accumulation-style recursion of the Halo lineage (the *Halo 2 Guide*, §“Recursive proof composition and accumulation schemes”; the Halo-specific device in §“Accumulation and the Halo trick”).

The intended proof system is Ragu, a Rust PCD framework implementing a modified Halo recursive SNARK with no trusted setup over the Pasta cycle. A working pipeline exists in code — application registration, **seed** (leaf proofs), **fuse** (binary merge), **rerandomize**, **verify**, with passing multi-node PCD-tree integration tests (`crates/ragu_pcd/src/{lib.rs,fuse/mod.rs,verify.rs}` at commit 830bbcd, 2026-07-05). The same code is explicit that it is pre-release: the **Proof** struct carries full polynomials and verification is linear-time (no inner-product opening argument or decider exists anywhere in the workspace), the Fiat–Shamir domain tag is the literal `b"FIXME"` (`crates/ragu_pcd/src/lib.rs:50--51`), and registry binding carries an in-code `FIXME(security)`. Downstream, the `zcash_tachyon` crate registers all nineteen proof-tree steps but compiles only against Ragu’s `mock` feature, with note commitments, nullifier derivation, and

proof creation/merge/verification “Stubbed (pending Ragu PCD and Poseidon integration)” (tachyon README.md at 26fdc165).

**Classification.** The Ragu and `zcash_tachyon` code facts are *specified* in the only sense available before a spec: verifiable source at a pinned commit. The wallet-state design — the headers, the nineteen steps, the fold — is *designed but unspecified*: it exists in the project book and nowhere at ZIP rigor, and a specification would need to pin the header encodings, the per-step relations, the Poseidon instances and domain tags, the GGM depth and arity, and the epoch length. The book’s soundness section is prose — no formal definitions, theorems, or machine-checked statements, and the eprint contains none either — so recursive soundness of the tree, and whether Ragu’s linear-time verifier can be carried to a succinct decider at the sizes Tachyon needs, are *open problems* (ragu crates/ragu\_pcd/src/verify.rs:122 records three missing verifier checks; the performance figures in the Ragu book are self-labelled rough estimates).

## 2.2 What the fold removes

Two cost centres of the deployed protocol disappear; both removals are properties of the *designed-but-unspecified* core above and inherit its bin except where noted.

**Trial decryption of the chain.** Nothing on today’s chain marks which Action pays a wallet, so detection is exhaustive: the wallet trial-decrypts every Orchard output from its birthday height, and restore-from-seed replays that scan over the entire chain since the birthday (*Ironwood Guide*, §“Transmission and detection of a note: encryption, trial decryption, and synchronisation”). Under Tachyon the wallet never scans. Detection and delivery move to the payment channel (§2.4), and the absence-tracking that synchronisation actually requires is delegated to a service that works only on the opaque nullifier windows of §2.1: the eprint asserts services are “fully oblivious to their clients’ transaction details” (eprint 2025/2031), and the book’s role table assigns the service the `Unspent` steps beside the note-free anchor-chain segments, and none that touch the note (`src/proof-tree.md` at 26fdc165). Groundwork exists at ZIP rigor: ZIP 2004, “Remove the dependency of consensus on note encryption” (Status: Draft at c6b70358), makes the consensus layer independent of the ciphertexts a delivery redesign would drop, though its `ephemeralKey` handling is an in-file TODO. No formal definition of “oblivious” exists — no threat model and no security definition — which we treat as an *open problem* (the trust model, §5.1.3; the timing-correlation attack the eprint calls potentially fatal, §5.2.1). The aggregate cost claim is likewise unproven: the designer’s estimate that total service effort is “asymptotically similar” to today’s validators is a forum statement (t/50789 #28, 2025-11-21) with no published complexity analysis, and no project benchmark exists — circulating throughput and size figures are analyst estimates (Messari; CoinDesk Research, 2026-06-30), not project figures.

**Per-note witness maintenance.** Today the wallet appends every published cmx to a pruned local shard tree, marks the positions of its own notes, stores a checkpoint per block, and rebuilds each authentication path on demand from retained subtree roots (*Ironwood Guide*, §“Proof of ownership of *some* valid note: the commitment tree”; the design of librustzcash’s `shardtree`). Tachyon has no note commitment tree. Pool state is a hash chain (§2.3); membership of the note’s creation is proven once, when `SpendableInit` checks `cm` against the creation stamp’s tachygram set; and the lineage thereafter carries no path — there are no siblings to refresh as others append, because there is no tree (book, `src/proof-tree.md`, `src/anchor.md` at 26fdc165). What remains is the forward obligation: the wallet must keep proving absence up to a recent anchor before it can spend, work proportional to its own note count and elapsed epochs rather than to chain traffic, and delegable as above. *Designed but unspecified* throughout: the creation-stamp membership argument and the anchor-rooting rule exist only in the book.



## 2.3 Validator-side consequences

A node today maintains two unbounded structures: the append-only commitment tree, from which it recognises anchors, and the ever-growing nullifier set, against which it checks each Action’s  $\text{nf}_{\text{old}}$  — the one check that cannot sit inside a proof, because it concerns global history (*Ironwood Guide*, §“What a node verifies, without ever observing the note”). The Tachyon design replaces both with one running Poseidon hash chain and bounds the duplicate check to a recent window.

Each stamp accepted into a block advances the pool state by

$$\text{anchor}' := \text{Poseidon}_{\text{Tachyon-StampFld}}(\text{anchor}, x, y),$$

where  $(x, y)$  are the coordinates of the stamp’s tachygram-set commitment; an empty block advances it by  $\text{Poseidon}_{\text{Tachyon-EmptyBlk}}(\text{anchor})$ ; and the crossing into epoch  $e + 1$  by  $\text{Poseidon}_{\text{Tachyon-EpochStp}}(\text{anchor}, e + 1)$ , the only domain that absorbs the epoch index (book, `src/anchor.md` at 26fdc165). Anchors are the end-of-block states of this chain; a spendable or stamp proves continuity from one anchor to a later one rather than membership in a tree.

The double-spend check becomes a windowed duplicate check: consensus checks every tachygram of a published stamp for duplicates over every block of the current and the immediately preceding epoch, rejects on a hit (book, `src/tachygrams.md` at 26fdc165), and may permanently prune tachygrams older than that window (forum t/50789 #22, 2025-11-04; the book states no pruning rule). Two epochs suffice in the design because a stamp’s anchor may have been lifted within its epoch (so the whole anchor epoch is scanned, not merely blocks after the anchor), and because a spend publishes the note’s nullifiers for both the anchor epoch  $e$  and epoch  $e + 1$  — the next-epoch nullifier published in  $e$  is exactly what a second spend of the same note would publish in  $e + 1$  (book, `src/tachygrams.md` at 26fdc165; the dual publication itself is in `src/nullifiers.md`).

**Classification.** The window rule and prunability are *designed but unspecified*: the window rule appears in the book only, prunability only in a forum statement (t/50789 #22, 2025-11-04); the eprint states just that “validators must still check for duplicates”; no consensus ZIP drafts the rule. A specification would need the epoch length — the parameter the entire pruning model hinges on, which the project itself marks as requiring empirical analysis (tachyon-zcash/tachyon issue #79) — and the normative force of end-of-block anchoring, on which the book’s own source wavers between “may”, “should”, and “must” in an inline comment (`src/anchor.md`). The consequence of pruning is an *open problem*: what validators discard, wallets and services still need, and the eprint concedes that the sudden shutdown of syncing services “could render the funds of all their dependent users unspendable, potentially permanently” (eprint 2025/2031, §1.4); no data-availability or incentive design exists (§5.2.2).

## 2.4 Payment delivery

Removing detection-by-trial-decryption reopens the question the deployed protocol answers with in-band secret distribution (*Ironwood Guide*, §“Transmission and detection of a note: encryption, trial decryption, and synchronisation”): how does the recipient learn of the note? The project’s answer changed in May 2026. We present both stages and date every statement; all quotations are verbatim from the cited artifacts.

**The out-of-band era (all statements predate 2026-05-26).** The design of “Tachyaction at a Distance” (Bowe, 2025-05-15) made Tachyon “out-of-band payments for the first time in a Zcash shielded protocol”: the sender delivers the note plaintext to the recipient over a channel outside the chain, and the on-chain ciphertext disappears. The simplification cascades through the key hierarchy — key diversification, viewing keys, and payment addresses are removed; the



note drops the diversifier and  $\rho$ ; the diversified transmission key  $\text{pk}_d$  gives way to a payment key deterministic per spending key (§3.2, which owns the construction; the book’s `src/keys.md` opens with the in-file warning “this is significantly outdated”). Unlinkability becomes the wallet layer’s problem, deferred to the URI substrate: ZIP 321 payment-request URIs and ZIP 324 URI-encapsulated payments. That substrate is *specified* only in part. ZIP 321 is Status: Active. ZIP 324 is Status: Draft, predates Tachyon and mentions it nowhere, declares that “It is outside the scope of this proposal to establish a secure communication channel”, derives its `rseed` under a domain separator that is the literal `0xFIXME`, is Sapling-only, and ends with a “Publication Blockers” section (zips `zip-0324.rst` at `c6b70358`). ZIP 231 memo bundles (Status: Draft) decouple memo data from individual outputs at full cryptographic rigor, but their v6 carrier was orphaned when ZIP 230 was withdrawn (zips `zip-0231.md`, `zip-0230.rst` at `c6b70358`). The out-of-band payment protocol itself was never specified at any rigor; the project roadmap listed two unreconciled delivery approaches (`tachyon.z.cash/roadmap/`, fetched 2026-07-08). The founding post had already conceded the cost: without in-band distribution “the user cannot rely on the blockchain as a storage mechanism for recovering their funds from a seed phrase” (Bowe, 2025-04-02).

**The pivot (2026-05-26).** Issue #121 of `tachyon-zcash/tachyon`, opened 2026-05-26 and still open as of 2026-07-08, replaces the out-of-band story in two sentences, which we quote whole: “Since we’re using PIR to make repeated payments flows without OOB, there are stale parts of the book that will need to change. We’re retaining in-band secret distribution for recover from seed flows.” Repeated-payment flows are thus to use private information retrieval (PIR), a query protocol in which the server learns nothing about which record the client fetched, rather than sender-to-recipient out-of-band delivery — the roadmap adds a post-quantum key exchange for the channel — while in-band secret distribution returns, scoped to recover-from-seed and restoring exactly the property the founding post conceded was lost. The protocol is “developed independently in collaboration with” the team and has no published specification; its public trace is a roadmap paragraph and podcast/whiteboard discussion (`tachyon.z.cash/roadmap/`; ZK Podcast 388, 2026-01-21). *Designed but unspecified* at its thinnest: a specification would need the PIR scheme and its database and server model, the key-exchange construction, the recover-from-seed format (which secrets stay in-band, under which keys, at what fee treatment), and the consequences for the key-hierarchy removals above — the book is flagged stale against the pivot by issue #121 itself, so which of `src/keys.md`’s removals survive is undetermined. The post-quantum claim also needs scoping: Bowe’s “Zcash and Quantum Computers” (2025-10-16) limits Tachyon’s quantum benefit to privacy — the harvest-now-decrypt-later threat — while ECC-based proof soundness remains quantum-vulnerable; whether retained in-band recovery ciphertexts reuse the ECC key agreement, and so retain its harvest-now exposure, is undetermined. Recoverability at large remains an *open problem* by the designer’s own account: “Getting this right will be challenging, but we simply have no choice but to do it” (forum `t/50789` #22, 2025-11-04); on ZK Podcast 388 (2026-01-21) the recovery infrastructure was still “a can of worms”. The single sentence of issue #121 is the entire written resolution to date.

### 3 The design as published

The preceding section reconstructed the architecture; this one fixes the design’s object model as the project book publishes it: the mdBook of `tachyon-zcash/tachyon` @ `26fdc165` (2026-07-07, single-committer; §1.3), the design’s most complete public statement. Each subsection states what the book fixes — exact formulas, fields, and domain strings — names the Orchard analogue in one contrast, and closes with the object’s bin under Definition 1.1 and what a specification must still pin down; mechanisms the architecture section constructs and gaps the status sections classify are cross-referenced, not restated. Every claim here is *designed but unspecified* by de-

fault. Where the `zcash_tachyon` crate at the same commit realizes an object, the realization is *specified* in the only sense available before a spec — verifiable source at a pinned commit (§4.3) — with the README split as the boundary: keys, value commitments, signatures, and bundle construction “implemented and tested”; note commitments, nullifier derivation, proofs, and stamp compression “Stubbed”. One caveat cuts across every Poseidon-derived object: the crate routes all Poseidon calls through Ragu’s mock sponge (`src/digest/poseidon.rs`), so derivation *structure* and domain tags are code-anchored while the outputs come from Ragu’s provisional `PoseidonFp` instantiation (width  $T = 5$ , rate 4; the mock sponge evaluates the real permutation out-of-circuit), not from any Tachyon-pinned Poseidon instance. (The book’s  $\mathbb{F}_p$ ,  $\mathbb{F}_q$  are the series’  $\mathbb{F}_{p\text{Pallas}}$ ,  $\mathbb{F}_{p\text{Vesta}}$ .)

Staleness is stated once, here. The book post-dates the May-2026 payment pivot (issue #121; §1.3), yet two chapters carry the in-file marker `<!-- todo: this is significantly outdated -->` (`src/keys.md` line 3, whole chapter; `src/authorization.md` line 228, its Detailed Sequence), and the key chapter’s out-of-band claims are pre-pivot statements, dated pre-2026-05-26 (§2.4, §5.1.2); we document the book as published and do not relitigate per object. Eight `src/SUMMARY.md` entries carry no file: the four section parents (Data Model, Cryptographic Primitives, Protocol, Network Roles — the last has a filed Overview child), the three network-role pages, and the testnet chapter. One terminological drift: the bundle chapter’s *tachystamp* is the *stamp* of the proof-tree chapter and the crate; we write *stamp*.

### 3.1 Bundles and tachygrams

**Definition 3.1** (Tachyon bundle objects). A *tachygram* is a 32-byte field element representing either a nullifier or a note commitment; consensus does not distinguish the two. A *tachyaction* indistinguishably represents the creation or destruction of a note — a commitment tachygram proves creation, a nullifier tachygram destruction — and is cryptographically bound to one tachygram that it does *not* contain. It contains `cv`, a 32-byte homomorphic commitment to the created or destroyed value; `rk`, a 32-byte public key bound to one tachygram; and `sig`, a 64-byte RedPallas signature by `rk` over the transaction sighash. A *stamp* contains an anchor, the recursive proof (which may be aggregated), and the covered actions’ tachygrams; the proof establishes that each tachygram creates a new note or destroys an existing one, that tachygrams are correctly bound to action keys, and that the action balance effect matches the pool balance effect. A *bundle* collects the tachyactions, the integer net pool effect `value_balance`, a binding signature over the same sighash as the action signatures, and the stamp (book, `src/bundle.md` @ 26fdc165).

The covering proof witnesses a commitment to the *unordered set* of the stamp’s tachygrams as a root polynomial,  $\text{tachygram\_acc}(X) = \prod_i (X - \text{tg}_i)$ , and this set is the atomic unit that folds into pool state to contribute to the anchor (book, `src/tachygrams.md` @ 26fdc165); the fold and the two-epoch duplicate scan the set feeds are constructed at §2.3.

**Wire format.** The first byte, `tachyonBundleState`, selects one of three states: `0x00` non-tachyon (no bundle), `0x01` stamped, `0x02` stripped; the body layout (value balance as `i64`, the `(cv, rk)` action vector, per-action and binding signatures) is fixed only in the crate’s doc tables (`crates/tachyon/src/bundle/mod.rs`). The stamp trailer (state `0x01`) carries `cActionsTachyon` (32 bytes, a “compressed Pedersen commitment” to the stamp’s merged action-digest set; §3.6), `anchorTachyon` (32 bytes), `nTachygrams` (`compactsize`), `vTachygrams` (`32n` bytes), and `proofTachyon`, a `PROOF_SIZE` blob — “serialized proof of fixed size”. The stripped trailer (state `0x02`) carries only `tachyonAggregateId`, the 64-byte nonzero `wtxid` (the ZIP 239 identifier `txid || auth_digest`; §3.8) of the covering aggregate: the all-zero `wtxid`, naming no aggregate, is rejected on read even for a stripped bundle whose action vector is empty, and the stripped trailer carries no `cActionsTachyon` — that field rides on the stamp and strips away when a bundle becomes an adjunct.

The Orchard contrast: an Orchard Action publishes typed, distinguishable fields ( $\text{cv}^{\text{net}}$ ,  $\text{nf}^{\text{old}}$ ,  $\text{rk}$ ,  $\text{cmx}$ ) plus note ciphertexts, with one per-transaction Halo 2 proof (the *Ironwood Guide*, §“What a node verifies, without ever observing the note”); a tachyaction is single-effect, ciphertext-free, does not carry its tachygram, and its proof is strippable and aggregatable across transactions — a lifecycle with no Orchard analogue.

**Classification.** *Designed but unspecified*, with the wire format’s structure specified-as-code: the crate’s doc tables mirror the book’s byte-for-byte and the nonzero-`wtxid` rule is enforced on read (`bundle/mod.rs`, `stamp/mod.rs`). A specification must still pin `PROOF_SIZE`, which no number in the book fixes (the crate defers to Ragu’s `PROOF_SIZE_COMPRESSED`), and the point-compression convention, fixed only informally per context (the trailer compresses; the anchor absorbs complete coordinates, §3.5).

### 3.2 The key hierarchy

**Definition 3.2** (Tachyon key hierarchy). The spending key  $\text{sk}$  is raw 32-byte entropy, matching Orchard’s representation and preserving the full 256-bit key space. With  $\text{PRFexpand}_{\text{sk}}(t) = \text{BLAKE2b-512}(\text{Zcash\_ExpandSeed}, \text{sk} \parallel t)$ , the book derives

$$\text{ask} = \text{ToScalar}(\text{PRFexpand}_{\text{sk}}([0\text{x}21])), \quad \text{nk} = \text{ToBase}(\text{PRFexpand}_{\text{sk}}([0\text{x}22])),$$

a long-lived scalar in  $\mathbb{F}_{p_{\text{Vesta}}}$  and a base-field element in  $\mathbb{F}_{p_{\text{Pallas}}}$ , with  $\text{ak} = [\text{ask}] \mathcal{G}$ ; the payment key is

$$\text{pk} = \text{Poseidon}(\text{PK\_DOMAIN}, \text{ak}_x, \text{nk}),$$

with  $\text{ak}_x$  the  $x$ -coordinate of  $\text{ak}$  — the chapter leaves `PK_DOMAIN` symbolic; only the domain-separator table fixes it as `Tachyon-PkDerive` (§3.8). The *proof authorizing key*  $\text{pak} = (\text{ak}, \text{nk})$  enables proof construction without spend authority; the per-note delegation bundle  $(\text{ak}, \text{mk})$  restricts a prover to one note, all epochs, where  $\text{mk}$  is the note’s GGM master key (§3.4). (Book, `src/keys.md` @ 26fdc165.)

**Sign normalization and tiers.** RedPallas requires  $\text{ak} = [\text{ask}] \mathcal{G}$  to have  $\tilde{y} = 0$  (bit 255 of the compressed encoding); if  $\tilde{y}(\text{ak}) = 1$  the derivation negates  $\text{ask}$  ( $[-\text{ask}] \mathcal{G} = -[\text{ask}] \mathcal{G}$ ), once, at derivation time (`src/keys.md`; `crates/tachyon/src/keys/private.rs` lines 83–98). The key  $\text{ask}$  cannot sign directly and  $\text{ak}$  cannot verify directly: both are randomized per action into  $\text{rsk}$  and  $\text{rk}$  (§3.7), so each on-chain  $\text{rk}$  is unlinkable to  $\text{ak}$ ; the book names the tiers private scalar  $\rightarrow$  circuit witness  $\rightarrow$  public on-chain (`SpendAuthorizingKey`, `SpendValidatingKey`, `ActionVerificationKey`). The payment key is *deterministic per spending key* — every note from one  $\text{sk}$  shares one  $\text{pk}$ , no per-note diversification — and since  $\text{pk}$  is committed in  $\text{cm}$ , accumulator membership transitively pins  $\text{ak}$  and  $\text{nk}$ : a wrong  $\text{nk}$  gives a wrong  $\text{pk}$ , a wrong  $\text{cm}$ , and inclusion fails.

The Orchard contrast: the root  $\text{sk}$ , the `PRFexpand` construction, the  $\text{ask} \rightarrow \text{ak}$  derivation with the same sign normalization, and RedPallas re-randomization survive (the *Ironwood Guide*, §“Keys: capabilities for spending and for viewing”; tag bytes differ, `0x06/0x07/0x08` against `0x21/0x22`), while  $\text{r}_{\text{ivk}}$ ,  $\text{fvk} = (\text{ak}, \text{nk}, \text{r}_{\text{ivk}})$ ,  $\text{ivk}$ ,  $\text{dk}$ ,  $\text{ovk}$ , the FF1-AES256 diversifier  $d$ ,  $\text{g}_d$ , and  $\text{pk}_d$  are removed, and  $\text{pk}$ ,  $\text{pak}$ , and the per-note  $(\text{ak}, \text{mk})$  bundle are new with no Orchard counterpart.

**Classification.** The derivations and normalization are *specified* as tested code in the crate’s key module, with  $\text{pk}$  computed through the mock sponge (`keys/private.rs`). The intent around  $\text{pk}$  — unlinkability deferred out-of-band — is the pre-pivot claim re-examined at §5.1.2. A specification must pin `PK_DOMAIN` normatively, the encoding of  $\text{ak}_x$ , and — after the pivot — which key-hierarchy removals survive.

### 3.3 Notes and commitments

**Definition 3.3** (Tachyon note). A note carries exactly four fields:  $\mathbf{pk} \in \mathbb{F}_{p_{\text{Pallas}}}$ , the recipient’s payment key;  $v$ , a u64 value in zatoshi with  $1 \leq v \leq 2.1 \times 10^{15}$ ;  $\psi \in \mathbb{F}_{p_{\text{Pallas}}}$ , the nullifier trapdoor; and  $\mathbf{rcm} \in \mathbb{F}_{p_{\text{Pallas}}}$ , the commitment trapdoor. Zero-value notes are forbidden. The commitment binds all four,

$$\mathbf{cm} = \text{Poseidon}_{\text{Tachyon-CmDerive}}(\mathbf{rcm}, \mathbf{pk}, v, \psi),$$

and is the published tachygram of an output action; a spend action publishes *two* nullifiers, at the present epoch  $e$  and the next epoch  $e + 1$ , as its tachygrams (book, `src/notes.md` @ 26fdc165; the commitment already appears in §2.1, the dual-nullifier rationale in §2.3).

The Orchard contrast: an Orchard note is a 5-tuple ( $\mathbf{addr} = (d, \mathbf{pk}_d), v, \rho, \psi, \mathbf{rcm}$ ) under a perfectly hiding Sinsemilla `NoteCommit`, with  $v \in [0, 2^{64})$  (the *Ironwood Guide*, §“Representation of a note: notes and note commitments”) and load-bearing zero-value dummies (*Ironwood Guide*, §“The Action statement, formally”); Tachyon drops the address and  $\rho$  entirely — “no diversifier, no rho, no unique value for faerie gold defense” since “the nullifier construction doesn’t require global uniqueness” (`crates/tachyon/src/note.rs`) — so where Orchard chains  $\rho^{\text{new}} := \mathbf{nf}^{\text{old}}$  (*Ironwood Guide*, §“Prevention of double-spending: nullifiers”),  $\psi$  reuse here harms only the owner (§5.1.1).

**Classification.** *Designed but unspecified*, and the commitment scheme is the book’s clearest internal churn marker: the note and domain-separator chapters fix Poseidon under `Tachyon-CmDerive`, while the crate’s module doc calls the scheme “TBD” — “(e.g. Sinsemilla, Poseidon)” pending what “is efficient inside Ragu circuits” (`note.rs`; cited at §5.1.1). A specification must pin the scheme and the field encodings; the  $\psi$ -uniqueness responsibility gap is classified at §5.1.1.

### 3.4 Epochs and the GGM nullifier tree

The derivation’s endpoints — the master key  $\mathbf{mk} = \text{Poseidon}_{\text{Tachyon-NfPrefix}}(\psi, \mathbf{nk})$ , the per-epoch leaf nullifier, the delegation window  $\delta$  with its sentinel, and the spendable lineage — are constructed at §2.1 and §5.1.1; none of that machinery appears in the eprint note, whose evolving-nullifier sketch is a flat PRF still taking  $\rho$  as input (quoted at §5.1.1). What the book fixes beyond those statements is the climb itself.

**Construction 3.4** (GGM climb). The tree has depth  $D$  and arity  $A$ , covering leaf epochs  $e \in [0, A^D - 1]$ ; a node at depth  $d$  covers a contiguous range of  $A^{D-d}$  consecutive epochs, and a node may only climb toward the leaves, never back toward the root — the property that makes absence-proof delegation trustless. To reach the leaf at epoch  $e$ , decompose  $e$  into  $d \in [0 \dots D]$  base- $A$  direction chunks, most significant first, and climb one step per chunk:

$$\text{KDF}_{\psi}^{\text{climb}}(e, 0) = \text{KDF}_{\psi}^{\text{root}}(\mathbf{nk}) = \mathbf{mk},$$

$$\text{KDF}_{\psi}^{\text{climb}}(e, d) = \text{Poseidon}_{\text{Tachyon-NfPrefix}}\left(\text{KDF}_{\psi}^{\text{climb}}(e, d-1), \left\lfloor e/A^{D-d} \right\rfloor \bmod A\right).$$

The value at  $\text{KDF}_{\psi}^{\text{climb}}(e, D)$  is the leaf key for epoch  $e$ ; the final `Tachyon-NfDerive` hash from leaf key to nullifier is constructed at §5.1.1 (book, `src/nullifiers.md` @ 26fdc165).

**Parameters.** The book leaves  $D$  and  $A$  symbolic everywhere. The crate pins working values: `GGM_TREE_ARITY` = 4, `GGM_CHUNK_SIZE` = 2 bits, `GGM_TREE_DEPTH` = 10 under the invariant  $A^D = \text{GGM\_MAX\_INDEX} + 1$  with `GGM\_MAX\_INDEX` = `EPOCH_MAX` =  $2^{20} - 1$  ( $4^{10} = 2^{20}$  epochs tile the index space exactly), and `EPOCH_SIZE` =  $2^{12} = 4096$  blocks, via a shift constant that drops to 4 under `cfg(test)` — placeholder values, not decisions (`crates/tachyon/src/keys/ggm.rs`,

`src/constants.rs`). Nothing in the book maps the epoch index to block height; the epoch length exists only in code, and its tuning is the open issue of §5.1.4.

The Orchard contrast: Orchard derives *one* nullifier per note,  $\text{nf} = \text{Extract}([F_{\text{nk}}(\rho) + \psi] \mathcal{G}^{\text{nf}} + \text{cm})$  — a Poseidon PRF plus a curve point — checked against the global, ever-growing nullifier set (the *Ironwood Guide*, §“Prevention of double-spending: nullifiers”); Tachyon replaces it with a per-epoch sequence, pure Poseidon with no curve point and no `Extract`, checked only over the two-epoch window of §2.3.

**Classification.** *Designed but unspecified*; the tree walk is code-anchored in structure (`keys/ggm.rs`) but its cryptography sits on the stubbed side of the README split. A specification must pin  $D$ ,  $A$ , the epoch length and activation semantics (§5.1.4), and the chunk encoding; the tip-nullifier exposure of the delegation flow is the open problem of §5.2.4.

### 3.5 Anchors, stamps, and the proof tree

The three anchor-chain formulas are constructed at §2.3; the unresolved normative force of end-of-block anchoring is classified at §5.1.5. The book adds two statements worth fixing here: intra-block states exist between stamp absorptions, and a proof such as `SpendableInit` “will produce a header that is likely at an intra-block state, and should be lifted to the end of a block by proving state continuity” (`src/anchor.md`; that the epoch step is the only domain absorbing the epoch index is already fixed at §2.3 — all cross-epoch binding rides on it); and absorbing the *complete*  $(x, y)$  coordinates of the stamp’s tachygram-set commitment, rather than a compressed encoding, makes the binding unambiguous — the crate sharpens this to a Vesta point whose  $\mathbb{F}_{p_{\text{Vesta}}}$  coordinates are each split into two 128-bit little-endian limbs, a six-element Poseidon absorption (`digest/poseidon.rs`; the sequence, tachygram-set, and action-set commitments all live on Vesta, `primitives/seq.rs`, `primitives/sets.rs`).

**The step graph.** Each proof step accepts arbitrary witness inputs and up to two PCD inputs, and emits a new PCD. The book fixes exactly nineteen steps over eight header types (Table 2) and a role matrix: the wallet runs every step touching the note’s commitment or master key; the sync service runs the anchor and `Unspent` steps over wallet-shared nullifier values, never note material (§5.1.3); the aggregator touches only published `StampHeaders` and the public anchor chain — the three anchor-chain steps to build lift segments, then `StampLift` and `MergeStamp` (book, `src/proof-tree.md` @ 26fdc165).

**Construction 3.5** (Sentinel-terminated sequence composition). An `Unspent`’s `elapsed` holds one nullifier coefficient per epoch-boundary crossing in its span, forward-chronological, terminated by a sentinel coefficient 1 at the crossing count  $s = \text{epoch\_end} - \text{epoch\_start}$ ; the sentinel keeps every committed sequence nonzero (an empty window is the constant 1), so the commitment never falls on the identity point, and it pins the sequence’s exact length. Step `NullifierStep` builds its single-leaf commitment homomorphically,  $[\text{nf}] \mathcal{G}_0 + \mathcal{G}_1$ , with  $\mathcal{G}_1$  carrying the sentinel. Composition is checked as polynomial identities at a Fiat–Shamir challenge:

$$\begin{aligned} C(X) &= L(X) + X^s (R(X) - 1) && (\text{UnspentFuse}), \\ C(X) &= L(X) + X^s (p - 1) + X^{s+1} R(X) && (\text{UnspentEpochFuse}), \\ R(X) &= E(X) + X^s (\text{nf\_end} - 1) + X^{s+1} && (\text{VerifyUnspent}), \end{aligned}$$

where  $p$  is the left half’s completing tip `nf_end` and  $E$  the `elapsed` sequence: in each identity the appended term overwrites the left sentinel and the trailing term re-terminates. The inputs  $L$ ,  $R$ , and  $p$  are bound by the recursive verification of the input PCDs *before* the challenge, and each identity is linear in them — which is what makes the splice sound. Step `MergeStamp` fuses two stamps at equal anchors by requiring, for each of the action-digest and tachygram multisets, that the merged set polynomial equal the *product* of the two inputs’ (book, `src/proof-tree.md` @ 26fdc165).

| Header          | Fields                                                                            |  |  |  |
|-----------------|-----------------------------------------------------------------------------------|--|--|--|
| AnchorChain     | (start, end)                                                                      |  |  |  |
| Unspent         | (anchor_prev, (epoch_start, nf_start), elapsed, (epoch_end, nf_end), anchor_last) |  |  |  |
| VerifiedUnspent | (cm, anchor_prev, (epoch_start, nf_start), (epoch_end, nf_end), anchor_last)      |  |  |  |
| NfPrefixHeader  | (cm, node, depth, index)                                                          |  |  |  |
| NullifierHeader | (cm, (epoch_start, nf_start), nf_seq_commit, (epoch_end, nf_end))                 |  |  |  |
| SpendableHeader | (cm, present_nf, anchor)                                                          |  |  |  |
| SpendHeader     | (cm, (cv, rk), present_nf, anchor)                                                |  |  |  |
| StampHeader     | (action_commit, tachygram_commit, anchor)                                         |  |  |  |

  

| Step                  | Left            | Right           | Witness                                                     | Output          |
|-----------------------|-----------------|-----------------|-------------------------------------------------------------|-----------------|
| AnchorSeed            | —               | —               | start, stamp_commit                                         | AnchorChain     |
| EmptyBlockSeed        | —               | —               | start                                                       | AnchorChain     |
| AnchorFuse            | AnchorChain     | AnchorChain     | —                                                           | AnchorChain     |
| UnspentSeed           | —               | —               | anchor_prev, (epoch, nf), stamp_tg_set                      | Unspent         |
| EmptyBlockUnspentSeed | —               | —               | anchor_prev, (epoch, nf)                                    | Unspent         |
| UnspentFuse           | Unspent         | Unspent         | left_elapsed_seq, combined_elapsed_seq, right_elapsed_seq   | Unspent         |
| UnspentEpochFuse      | Unspent         | Unspent         | left_elapsed_seq, combined_elapsed_seq, right_elapsed_seq   | Unspent         |
| VerifyUnspent         | Unspent         | NullifierHeader | elapsed_seq, nf_seq                                         | VerifiedUnspent |
| NfMasterSeed          | —               | —               | note, pak                                                   | NfPrefixHeader  |
| NfPrefixStep          | NfPrefixHeader  | —               | chunk                                                       | NfPrefixHeader  |
| NullifierStep         | NfPrefixHeader  | —               | —                                                           | NullifierHeader |
| NullifierFuse         | NullifierHeader | NullifierHeader | left_seq, merged_seq, right_seq                             | NullifierHeader |
| SpendableInit         | AnchorChain     | NullifierHeader | (pre_epoch_anchor, pre_cm_anchor), creation_set, present_nf | SpendableHeader |
| SpendableLift         | SpendableHeader | VerifiedUnspent | —                                                           | SpendableHeader |
| SpendBind             | SpendableHeader | —               | note, rcv, alpha, pak                                       | SpendHeader     |
| OutputStamp           | —               | —               | rcv, alpha, note, anchor                                    | StampHeader     |
| SpendStamp            | SpendHeader     | NullifierHeader | nf_next                                                     | StampHeader     |
| MergeStamp            | StampHeader     | StampHeader     | (action_set, tachygram_set) × left, merged, right           | StampHeader     |
| StampLift             | StampHeader     | AnchorChain     | —                                                           | StampHeader     |

Table 2: The eight proof-tree headers (top) and nineteen steps (bottom), fields verbatim (book, `src/proof-tree.md @ 26fdc165`). All nineteen steps are registered in the crate against Ragu’s mock (`crates/tachyon/src/stamp/proof/mod.rs`).



**Lineage and spend binding.** Step `SpendableInit` checks `cm` among the creation stamp’s tachygrams and roots the chain at `pre_epoch_anchor.next_epoch(epoch)`, pinning the starting GGM leaf index to the consensus epoch — without it, a note spent in its creation epoch crosses no boundary and the index is a free witness. On `present_nf` the book disagrees with itself: its walkthrough binds `present_nf` to the proven leaf by equality at init — matching the step’s `NullifierHeader` input in Table 2 — while its reference section calls it “unconstrained here”, pinned to a genuine leaf only by the first lift — another unresolved internal inconsistency at 26fdc165 (`src/proof-tree.md`). Step `SpendBind` requires `spendable.cm = note.commitment()` — rejecting a phantom note reusing  $\psi$  under a different value and `cm` — and its `SpendHeader` is consumed only by `SpendStamp`, so the note never propagates. Step `SpendStamp` composes it with a length-2 `NullifierHeader` range, witnesses `nf_next`, binds the published pair to the boundary leaves by equality, and guards both nullifiers nonzero (the guard’s rationale is quoted at §5.1.1); deferring this binding and the action digest here keeps each step within its per-step gate budget. The book’s privacy claim: the note’s age never becomes public — the lineage carries a single current nullifier, not a polynomial with a consumed offset.

**Stamps.** Step `OutputStamp` is the only stamp-producing step taking an anchor as direct witness (an output has no prior chain state); it rejects zero and over-range values, and the wallet typically anchors outputs at the spends’ height so `MergeStamp` proceeds without an intervening lift. Step `StampLift` advances a composed stamp’s anchor toward the tip before publication; validators reconstruct both set commitments from published bundles, check the proof against them, and confirm the anchor against the chain. An aggregated stamp has the same shape as any other and is eligible for further aggregation.

The Orchard contrast: Orchard proves membership on every spend via a 32-layer Sinsemilla Merkle path against a per-block tree anchor (the *Ironwood Guide*, §“Proof of ownership of *some* valid note: the commitment tree”); the no-tree, proven-once replacement is stated at §2.2.

**Classification.** *Designed but unspecified* throughout: the nineteen steps exist as circuit-shaped Rust against the mock (`stamp/proof/mod.rs`), not as mathematics, and the soundness section is prose (§2.1; §5.1.5). A specification must pin the header encodings, the per-step relations, the polynomial-commitment scheme and coefficient generators  $\mathcal{G}_i$  (the book’s `Commit` is abstract), and the Poseidon instance.

### 3.6 Aggregation

The lifecycle nomenclature is fixed in Table 3: wallets broadcast *autonomes*; aggregators strip and merge stamps into *aggregates*; miners strip covered transactions into *adjuncts*, and a block carrying an aggregate must include every covered transaction sans its now-redundant proof. Aggregates subdivide into *innocent* (no Tachyon actions of its own, e.g. other-pool-only activity) and *based* (proving its own actions, e.g. a coinbase aggregate paying miner rewards through a Tachyon action). The process is decompress–merge–recompress: select transactions; deserialize and validate stamps; match or update anchors (`StampLift`); merge stamps — tachygrams as exclusive sets, proofs recursively; serialize, compress, publish (book, `src/aggregation.md` @ 26fdc165).

| Term      | State    | Provenance | Validation                                       |
|-----------|----------|------------|--------------------------------------------------|
| autonome  | stamped  | wallet     | complete proof with all inputs                   |
| aggregate | stamped  | aggregator | merged proof needs input from other transactions |
| adjunct   | stripped | miner      | input to merged proof from another transaction   |

Table 3: Bundle lifecycle nomenclature (book, `src/aggregation.md` @ 26fdc165).



**The action-set indicator.** An aggregate’s stamp proves its own plus every covered adjunct’s actions but carries no proof header — verifiers reconstruct it from covered actions, which before block inclusion are not at hand. The stamp trailer therefore carries `cActionsTachyon`, an author-provided *assistive indicator* mirroring the action-set commitment the proof attests to, letting an observer cheaply tell an autonome from an aggregate and judge a collection of adjuncts complete without verifying the proof. It is authorization data, not effecting data — it rides the strippable trailer, contributes to `auth_digest`, and is absent from adjuncts — and *not* a soundness mechanism: the proof binds the action set regardless, and a wrong indicator only harms its author. Function `Stamp::verify` uses it as a fast-fail gate before expensive proof verification (book, `src/aggregation.md`; code, `stamp/mod.rs`).

The Orchard contrast: no analogue exists — an Orchard bundle’s proof is per-transaction, never stripped, never merged across transactions (the *Ironwood Guide*, §“What a node verifies, without ever observing the note”).

**Classification.** *Designed but unspecified*, shading into *open problem* inside the chapter’s own file: the tachygram set-union validation algorithm and the aggregation limits are in-file TODOs, and the closing comment lists undecided economics — aggregate size limited by commitment size before block size (“on the order of thousands of actions”), single-aggregate merging serial though multiple aggregates parallelize, ser/de expensive, and the optimal-aggregate, miner-integrate-vs-p2p, and selection-order questions open (`src/aggregation.md`, comments). The “Block Layout” section the transaction-identifiers chapter cites does not exist in `aggregation.md` at this commit — a dangling link. A specification must pin the union-validation algorithm, the limits, and the gossip protocol.

### 3.7 Authorization

A bundle carries three authorization layers: per-action signatures binding each tachyaction to its tachygram, value commitments hiding individual values while preserving their algebraic sum, and a binding signature proving the declared balance (book, `src/authorization.md @ 26fdc165`).

**Construction 3.6** (Deterministic action randomizer). The planner selects arbitrary per-action entropy  $\theta$ ; the custody device receives each note and  $\theta$  to independently confirm the planning work. The randomizer is deterministic from  $(\theta, \text{cm})$  under per-effect personalizations:

$$\alpha_{\text{spend}} = \text{BLAKE2b-512}_{\text{Tachyon-Spend}}(\theta \parallel \text{cm}), \quad \alpha_{\text{output}} = \text{BLAKE2b-512}_{\text{Tachyon-Output}}(\theta \parallel \text{cm}).$$

Every action signs with a unique `rsk`, published as `rk = [rsk]G`. For a spend,

$$\text{rsk} = \text{ask} + \alpha, \quad \text{rk} = [\text{ask} + \alpha]G = \text{ak} + [\alpha]G,$$

so the planning device derives `rk` from the validating `ak` without ever holding `ask`, and custody confirms `rk` before signing. For an output, `rsk =  $\alpha$`  and `rk =  $[\alpha]G$` : no authority is needed. Because  $\alpha$  derives from  $\theta \parallel \text{cm}$ , the key `rk` is itself a commitment to `cm`, and the signature transitively binds each action to its tachygram without the tachygram appearing in the action.

**Bundle commitment and sighash.** The bundle’s effect digest accumulates per-action digests into a set polynomial:

$$d_i = \text{Poseidon}_{\text{Tachyon-ActionDg}}(\text{cv}_i \parallel \text{rk}_i), \quad \text{action\_acc} = \text{Commit}\left(\prod_i (X - d_i)\right),$$

hashed as `BLAKE2b-512ZTxIdTachyonHash(action_acc || value_balance)`. The same action-digest accumulation is bound to the stamp proof; the stamp itself is excluded from the bundle commitment because it strips during aggregation. All signatures — action and binding — sign one transaction-wide sighash, computed at the transaction layer from every pool’s bundle commitment and passed

to the Tachyon layer as opaque bytes. Consensus recomputes the action digests from the visible actions and checks them against both the sighash (via the bundle commitment) and the stamp (via the polynomial commitment in the PCD header); a modified action breaks both checks.

**Value balance.** The value commitment and binding algebra are Orchard’s, deliberately: with  $v$  signed (positive spends, negative outputs) and  $\text{rcv}$  random in  $\mathbb{F}_{p_{\text{Vesta}}}$ ,

$$\text{cv} = [v] \mathcal{V} + [\text{rcv}] \mathcal{R}, \quad \text{bvk} = \sum_i \text{cv}_i - [\text{value\_balance}] \mathcal{V} = \left[ \sum_i v_i - \text{value\_balance} \right] \mathcal{V} + [\text{bsk}] \mathcal{R},$$

where  $\text{bsk} = \sum_i \text{rcv}_i$ : the planner signs the sighash with  $\text{bsk}$  directly, without custody assistance, and validators’ reconstructed  $\text{bvk}$  equals  $[\text{bsk}] \mathcal{R}$  exactly when  $\sum_i v_i = \text{value\_balance}$ . The generators are *shared with Orchard*, from the domain `z.cash:Orchard-cv`. Two footnotes flag churn: the  $\text{rcv}$  derivation “may be revised in the future to incorporate a hash of the note commitment”, and the binding signature uses `reddsa::orchard::Binding`, hardcoding  $\mathcal{R}$  — “We should consider defining a unique personalization.”

The Orchard contrast: the value machinery is identical by construction — same Pedersen  $\text{cv}$ , same  $\text{bvk}$  equation, literally the same generators (the *Ironwood Guide*, §“Conservation of value: value commitments and the binding signature”) — while the randomizer differs in kind: Orchard draws  $\alpha$  uniformly at random per Action with no planner/custody split (*Ironwood Guide*, §“Authorisation of the spend: RedPallas”), and every Orchard Action is a paired spend-plus-output carrying one `SpendAuthSig` (*Ironwood Guide*, §“A single shielded payment, end to end”; §“What a node verifies, without ever observing the note”).

**Classification.** This is the *specified-as-code* side of the README split: the randomizer, re-randomized keys, value commitments, and both signature layers are implemented and tested (`entropy.rs`, `keys/private.rs`, `value.rs`, `reddsa.rs`). Unpinned by any artifact: the sighash construction (opaque bytes at this layer), the  $\text{rcv}$  derivation, the generator personalization, the Commit scheme behind `action_acc`, and — flagged in-file — whether  $\alpha$  moves to a curve-native derivation (`src/domain-separators.md`, comment).

### 3.8 Transaction identifiers and domain separators

A bundle’s authorization form changes through aggregation — stamping, merging, and stripping produce bit-different authorizations of the same effecting data — and `wtxid = txid || auth_digest` (ZIP 239) must distinguish the forms. Tachyon routes every mutable part through `auth_digest`: `txid` commits to effecting data only, `action_acc || value_balance`, so stripping, merging, and re-stamping leave it unchanged, while the authorization side commits to the signatures plus the stamp trailer:

$$\begin{aligned} \text{auth\_digest\_contribution} &= \text{BLAKE2b-256}_{\text{ZTxAuthTachyHash}}(\text{vActionSigs} || \text{bindingSig} || T), \\ T &= \begin{cases} \text{cActionsTachyon}_{32} || \text{anchor}_{32} || \text{vTachygrams} || \text{proof} & \text{if stamped} \\ \text{stampWtxid}_{64} & \text{if stripped} \end{cases} \end{aligned}$$

(book, `src/transaction-identifiers.md` @ 26fdc165). Each physical form thus yields a distinct `wtxid`. The personalization `ZTxAuthTachyHash` is declared “a placeholder until a Tachyon-ZIP amendment to ZIP-244 fixes it.” An adjunct’s covering-aggregate reference is a `wtxid`, assigned by the miner at block assembly; the covering aggregate must itself be top-level in the block — never stripped, never further aggregated — so the reference is stable.

**A book-internal (and book-code) discrepancy.** The transaction-identifiers formula writes `BLAKE2b-256` for the `auth_digest` contribution — matching ZIP 244’s digests —

while the book’s own domain-separator table (Table 4) and the crate (`hash_length(64); digest/blake2b.rs`) both say `BLAKE2b-512`: the hash width is an unresolved encoding at `26fdc165`. The code also fixes encodings the book leaves loose: the bundle commitment hashes the action commitment’s complete coordinates  $x \parallel y$  then `value_balance` as a little-endian i64, and a transaction with no Tachyon bundle contributes the personalized hash of the *empty* string — documented as distinct both from a stripped bundle and from a bundle with no actions and zero balance (`blake2b.rs`, `bundle/mod.rs`).

The Orchard contrast: the construction mirrors the ZIP-244 personalized-BLAKE2b pattern of `ZTxIdOrchardHash` (the *Ironwood Guide*, §“Conservation of value: value commitments and the binding signature”, paragraph “The transaction digest (ZIP-244)”); the novelty is that the entire stamp rides `auth_digest`, keeping `txid` invariant under aggregation.

**Domain separators.** Table 4 is the book’s complete table — its most specification-like artifact, byte-exact strings each realized as a code constant. The crate fixes the in-circuit realization the book omits: every Poseidon domain tag is a 16-byte ASCII string absorbed as the *first* sponge input, embedded as  $\mathbb{F}_{p_{\text{Pallas}}}$  via `Fp::from_u128(u128::from_le_bytes(tag))`, with arities: action digest 5 (domain, cv and rk as full coordinate pairs), payment key 3, note commitment 5, master key 3, prefix step 3, nullifier 2, anchor stamp step 6, anchor empty step 2, anchor epoch step 3 (`digest/poseidon.rs`) — all through the mock sponge.

| Purpose                        | Hash          | Value                          |
|--------------------------------|---------------|--------------------------------|
| Spend $\alpha$                 | BLAKE2b-512   | Tachyon-Spend                  |
| Output $\alpha$                | BLAKE2b-512   | Tachyon-Output                 |
| Bundle commitment              | BLAKE2b-512   | ZTxIdTachyonHash               |
| Bundle auth digest             | BLAKE2b-512   | ZTxAuthTachyHash               |
| PRF expand (0x21 ask, 0x22 nk) | BLAKE2b-512   | Zcash_ExpandSeed               |
| Nullifier prefix key           | Poseidon      | Tachyon-NfPrefix               |
| Nullifier derivation           | Poseidon      | Tachyon-NfDerive               |
| Note commitment                | Poseidon      | Tachyon-CmDerive               |
| Action digest                  | Poseidon      | Tachyon-ActionDg               |
| Payment key derivation         | Poseidon      | Tachyon-PkDerive               |
| Anchor stamp step              | Poseidon      | Tachyon-StampFld               |
| Anchor empty step              | Poseidon      | Tachyon-EmptyBlk               |
| Anchor epoch step              | Poseidon      | Tachyon-EpochStp               |
| Value commitment               | hash-to-curve | <code>z.cash:Orchard-cv</code> |

Table 4: Complete domain-separator table (book, `src/domain-separators.md` @ `26fdc165`). The auth-digest hash width is the discrepancy noted in the text: the book’s transaction-identifiers formula says `BLAKE2b-256`; this table and the crate say `BLAKE2b-512`.

**Classification.** *Designed but unspecified*, at the highest concreteness the corpus reaches: byte-exact strings with code constants, yet two entries are self-declared placeholders — `ZTxAuthTachyHash`, and the value-commitment generator borrowed from Orchard’s `z.cash:Orchard-cv` — and one is internally inconsistent (the 256/512 width, above). A specification must fix the hash widths, the personalization bytes, and above all the Poseidon instance the eight domains parameterize — parameters, rate, and capacity exist in no Tachyon artifact (the crate computes every domain through Ragu’s mock sponge, inheriting its provisional `PoseidonFp`, width  $T = 5$ , rate 4).

Taken together, the book at `26fdc165` fixes an object model at a concreteness no other Tachyon artifact approaches, and still leaves every load-bearing parameter symbolic: the GGM shape and epoch length (§5.1.4), the proof size, the Poseidon instance, the commitment scheme and its

generators, the sighash, and the note-commitment scheme itself. The sections that follow classify these gaps severally; the open problems are the design’s own (§5.2).

## 4 What is specified today

Every claim in this volume is classified *specified*, *designed but unspecified*, or *open problem*. This section is the specified bin, and it is exhaustive: a statement appears here only if a verifiable artifact backs it — a ZIP with a recorded status in `zcash/zips`, or source code read at a pinned commit. Our ground truth is `zcash/zips` at commit `c6b70358` (2026-07-05), `tachyon-zcash/ragu` at `830bbcd` (2026-07-05), and `tachyon-zcash/tachyon` at `26fdc165` (2026-07-07); repository metadata (releases, package registries, continuous integration) was queried on 2026-07-08. The two lower bins follow in §5.1 (designed but unspecified) and §5.2 (open problems).

The headline fact is negative. No component of the Tachyon core — the shielded protocol, the bundle format, the accumulator, the aggregator, oblivious synchronization — is specified anywhere: upstream `zcash/zips` contains no Tachyon ZIP or pull request, and the five core ZIP stubs in the project’s own tracker are empty files (`book/src/zips/` @ `26fdc165`; `tachyon-zcash/tachyon` issue #114). What the specified bin actually contains is periphery: one Active payment-request substrate, four groundwork drafts in varying states of health, and the code state of the proof layer and the protocol crate. We inventory each in turn.

### 4.1 ZIP 321 as substrate, and the four groundwork drafts

All ZIP facts below are read from `zcash/zips` @ `c6b70358`; line numbers refer to the files at that commit.

**ZIP 321 — Payment Request URIs (Status: Active).** This is the only entry at deployed rigor. It defines the `zcash:` URI scheme by which a payer is asked for payment: a grammar of `address`, `amount`, `memo` (base64url), `label`, and `message` parameters, with numeric parameter indices for multi-recipient requests (`zip-0321.rst`). Its role here is substrate: the pre-pivot Tachyon design removes per-note key diversification, and the design book assigns per-payment unlinkability to “out-of-band payment protocols (ZIP 321 payment requests, ZIP 324 URI encapsulated payments)” (`tachyon book/src/keys.md` @ `26fdc165`).

**ZIP 324 — URI-Encapsulated Payments (Status: Draft).** Created 2019-07-17, the draft predates Tachyon by six years and mentions it nowhere. It specifies sending a payment through any secure messaging channel as a URI encoding the secret spending key of an ephemeral, freshly funded address; anyone who learns the URI finalizes the payment by sweeping the encapsulated funds into their own wallet (`zip-0324.rst`, Abstract). Three limits bound what it pins down. First, the construction is Sapling-only: the encapsulated note is a Sapling note, keys derive under the path `m_Sapling/324' /...`, and Orchard appears zero times. Second, core details are unpinned: the master-key derivation `rseed = PRFexpandskm(·)` carries the literal placeholder `[0xFIXME]` as its domain separator (line 148), the note-identification section is marked “out of date ... To be revised” (line 384), and the draft closes with a “Publication Blockers” section (line 458). Third, the hard part is out of scope by declaration: “It is outside the scope of this proposal to establish a secure communication channel for transmission of URI-Encapsulated Payments” (line 107) — the unlinkability Tachyon defers to this document is thus deferred again, to the messaging layer.

**ZIP 231 — Memo Bundles (Status: Draft).** The cryptographic core is at full ZIP rigor. Memos leave the note plaintext and move into a per-transaction *memo bundle*: a sequence of 272-byte encrypted chunks capped at `memo_chunk_limit` × 256 = 16384 bytes of plaintext (16 KiB) per transaction (lines 209–216); each chunk sealed with ChaCha20-Poly1305 in a variant of the STREAM construction, the final chunk distinguished by a nonce suffix (lines 264–274); per-output

32-byte memo keys  $K^{\text{memo}}$  with two reserved values — all-0x00 for publicly decryptable, all-0xFF for no memo (lines 222–231); chunks of all memos interleaved by an order-preserving shuffle that MUST reveal to a recipient only the size of their own memo and the total count of chunks they cannot decrypt (lines 282–297); a two-pass decryption algorithm, one pass under counter nonces and one under the final-chunk nonce (lines 323–351); independent prunability via `fAllPruned`, with relay of pruned transactions forbidden (lines 391–434); and a fee treatment under ZIP 317 Revision 2, two free chunks whenever the transaction has shielded outputs (line 422 ff.). The gaps are equally explicit: the sighash commitment is “TODO: finish this as a modification to ZIP 248” (line 417); the lead byte is the unassigned placeholder `{MBLEADBYTE}` (lines 490, 588); and the carrier is orphaned — the transaction fields that would hold a memo bundle were defined by the withdrawn ZIP 230, and their designated new home, ZIP 248, is unmerged.

**ZIP 230 — Version 6 Transaction Format (Status: Withdrawn).** The draft carried the byte-precise OrchardZSA v6 wire format: Action Groups, the ZSA issuance bundle, the memo-bundle carrier fields, an explicit `fee` field, `zip233Amount`, and per-transparent-input `vSighashInfo` (`zip-0230.rst`, field tables). Its withdrawal warning states that “Transaction version number 6 is now defined by ZIP 248” (lines 39–40) — already stale, since ZIP 229 (Status: Draft, created 2026-06-13) now defines v6 for the NU6.3 pool and declares as non-requirements exactly the withdrawn machinery: ZSAs, `zip233Amount`, the explicit `fee` field, per-input sighash information, and ZIP 248’s “other extensibility affordances” (`zip-0229.md`, Non-requirements). The ZIP 230 byte format survives only as reference text with no deployment path.

**ZIP 246 — Digests for the Version 6 Transaction Format (Status: Withdrawn).** Created 2025-02-12 and withdrawn 2026-06-24, the draft defined v6 transaction identifiers, authorizing-data commitments, and signature digests; its withdrawal note reassigns all three to ZIP 229 (lines 39–41). Its one surviving idea is sighash-algorithm versioning: a closed per-transaction-version set of sighash algorithms, selected via `sighashInfo = [sighashVersion] || associatedData` (lines 106–123). The concept now lives only in withdrawn text and in the unmerged ZIP 248 pull request (zcash/zips#1156, idle since 2026-05-26); ZIP 229 itself hashes in the plain ZIP-244 style.

The pattern across the four drafts is uniform. Each piece of groundwork either lacks its carrier (ZIP 231), was withdrawn in favor of a narrower NU6.3 format (ZIPs 230 and 246), or defers its hardest component to another layer (ZIP 324). The single live v6 specification today is ZIP 229; the extensible-format home that ZIPs 230, 231, and 246 all defer to, ZIP 248, exists only as an open pull request.

## 4.2 Ragu at commit 830bbcd: the proof layer as code

Ragu is the proof-carrying-data (PCD) framework on which Tachyon’s wallet-state proofs are to run. No protocol specification exists: every protocol chapter of the Ragu book — assumptions, arithmetization, NARK, split accumulation, registry polynomial, recursion, analysis — is marked `<!-- todo -->` in the book index (`book/src/SUMMARY.md`). At this commit the code is therefore the only specification of what Ragu does, and we read the code.

**What the pipeline implements.** The repository is a `no_std` Rust workspace (version 0.0.0, edition 2024, MSRV 1.90) implementing a modified version of the Halo recursive-SNARK construction, with no trusted setup (`README.md`, `Cargo.toml`). The Pasta cycle is the only implemented curve cycle: crate `ragu_pasta` provides the sole `Cycle` implementation, with the Pallas base field as the circuit field (`crates/ragu_pasta/src/lib.rs`). A working PCD pipeline exists end to end: an application registers its steps through `ApplicationBuilder::register` and finalizes; seeding creates a leaf; `fuse` merges two proof-carrying nodes into one through an eleven-stage prover pipeline (files `_01_application.rs`

through `_11_circuits.rs` under `crates/ragu_pcd/src/fuse/`); `rerandomize` and `verify` complete the interface. Claims are tracked in two registries, native (Pallas-field) and nested (Vesta-field, CycleFold-style). An integration test builds and verifies a multi-node PCD tree (`crates/ragu_pcd/tests/nontrivial.rs`).

**Verification is linear-time; no opening argument exists.** Proofs are not succinct. Type `Proof` carries the full sparse polynomials of the argument — fifteen native polynomials plus the bridge polynomials, some twenty in all (`ragu_pcd/src/proof/mod.rs`). Function `verify()` checks a proof by sampling challenges, rebuilding the revdot claim polynomials, and evaluating the carried polynomials directly (`ragu_pcd/src/verify.rs`); an in-code note records that the native *c*-claim is tautological in the verifier and meaningful only inside the circuit. No inner-product-argument opening, decider, or proof-compression step exists anywhere in the workspace. Verification therefore runs in time linear in the proof, and the succinct final verifier Tachyon needs is future work, not present code.

**Soundness placeholders.** Three placeholders sit on the main branch, each self-documented. The Fiat–Shamir domain-separation tag is the literal `RAGU_TAG: &[u8] = b"FIXME"`, beneath the comment “choose a permanent domain separation tag before release” (`ragu_pcd/src/lib.rs`, lines 50–51). Registry binding — the mechanism that replaces per-circuit verification keys — runs six hash iterations from the placeholder “nothing-up-my-sleeve” challenges  $w = 2$ ,  $x = 3$ ,  $y = 5$ , under an in-code `FIXME(security)` stating that six iterations “is insufficient to fully bind the registry polynomial” (`ragu_circuits/src/registry.rs`, lines 673–692; issues #78, #316). The verifier, finally, omits three checks recorded in an in-code `TODO: registry_wx0_poly`, `registry_wx1_poly`, and `registry_wy_poly` (`verify.rs`, lines 122–124). Beyond soundness, zero knowledge is opt-in rather than default: the documentation of `fuse()` warns that accepting an RNG “is not an indication that the resulting proof-carrying data is zero-knowledge; that must be ensured by performing `Application::rerandomize` at a later point” (`fuse/mod.rs`), and `rerandomize` itself seeds a trivial proof under a comment reading “this is a temporary hack” (`lib.rs`, line 262).

**Assurance machinery.** The engineering hygiene around this pre-release core is heavy and, at the pinned commit, green. The workspace carries 445 `#[test]` functions and twelve property-testing suites; line coverage stood at 93.8% (Codecov snapshot, 2026-03-08). Twenty libFuzzer targets — witness generation and cheating, revdot and algebraic identities, Poseidon differentials, metamorphic driver behavior, verifier rejection, among others — run on a twice-weekly cron (`qa/fuzz/fuzz_targets/`, `.github/workflows/fuzz-cron.yml`: Sundays and Wednesdays). A Lean 4 formalization covers twenty-seven gadget-circuit files, and CI checks that the Lean sources extracted from the Rust gadgets are current (`qa/lean/Ragu/Circuits/`, `.github/workflows/fv.yml`). Supply-chain review runs under `cargo-vet` (`qa/supply-chain/`) and workflow linting under `zizmor`. All continuous-integration workflows were green at 830bbcd4 (GitHub Actions, queried 2026-07-08).

**What is absent.** No security audit exists: the README warns, verbatim, “Ragu is under heavy development and has not undergone auditing. Do not use this software in production.” No benchmark has been published: a bench harness runs in CI (`.github/workflows/bench.yml`) but publishes no numbers, and the only performance figures anywhere in the project are the Ragu book’s self-labelled rough estimates — which belong to the designed-but-unspecified bin, not to this one. No release exists: the workspace version is 0.0.0, the repository has zero GitHub releases, and no `ragu` crate is on crates.io (registry queries, 2026-07-08).



### 4.3 The `zcash_tachyon` crate: integration against a mock

Crate `zcash_tachyon` (`tachyon-zcash/tachyon @ 26fdc165`) implements and tests the conventional cryptographic plumbing: key derivation via `PRFexpand` with sign normalization, Pedersen value commitments over Pallas sharing Orchard’s generators, randomized RedPallas spend-authorization signatures, binding signatures, and bundle construction (`README.md`). Everything proof-shaped is stubbed. The crate consumes Ragu exclusively through its `mock` feature, pinning the dependency to a contributor fork (`github.com/turbocrime/ragu`, rev `98a15d32`) with features `mock` and `legacy-deps` (`crates/tachyon/Cargo.toml`). The mock is, in its own words, an “API-level mock of `ragu_pcd`” that “mirrors the shape of the real `ragu_pcd` API so downstream consumers (e.g. Zebra) can integrate against it ahead of the real implementation” (`ragu src/mock/mod.rs @ 830bbcda`); it performs no proof cryptography, though its `Sponge` does evaluate the real ragu Poseidon permutation out-of-circuit. Against this interface the crate registers its nineteen proof steps (`crates/tachyon/src/stamp/proof/mod.rs`). The README’s own stub list is exact: note commitments, nullifier derivation (the GGM tree PRF), proof creation, merge, and verification, and stamp compression/decompression all await “Ragu PCD and Poseidon integration”. The shape of the integration is therefore specified by code; its cryptographic content is not.

Taken together, the specified bin holds one Active URI substrate; one out-of-band payment draft that is Sapling-only with its core derivation unpinned; one memo-bundle draft whose cryptography is complete but whose carrier is orphaned; two withdrawn format ZIPs surviving as reference text; a proof framework whose pipeline runs but whose verifier is linear-time, incomplete, and Fiat–Shamir-tagged `b"FIXME"`; and a protocol crate integrating against a no-crypto mock. Every load-bearing Tachyon design — evolving nullifiers, the proof tree, aggregation, oblivious synchronization — lives outside this bin, in the sections that follow.

## 5 Designed but unspecified, and open problems

This section carries the two lower bins of the classification of Definition 1.1: constructions that exist in an informal source but in no artifact at ZIP or protocol-specification rigor, and problems the designers themselves acknowledge as unsolved. Nothing here is normative, and the global fact that forces both bins is established in §4: upstream `zcash/zips` (commit `c6b70358`, 2026-07-05) contains zero Tachyon ZIPs or pull requests, and the five core ZIP pages in the project’s own design book — shielded protocol, bundle, accumulator, aggregator, and oblivious syncing service — are eleven-line skeletons with empty *Dependencies*, *Design Considerations*, and *ZIP Draft* headings (`book/src/zips/ @ 26fdc165`, 2026-07-07).

For every entry we state three things: *what exists*, with the source and commit (source pinning as in §1.3; the mechanisms themselves are constructed in §2); *what a specification must pin down* before the entry can move to the specified bin (for open problems, what a resolution must establish); and *what would falsify the design* — the observation that would not merely delay the entry but refute it.

### 5.1 Designed but unspecified

#### 5.1.1 Nullifier evolution and re-randomization for oblivious synchronisation

**What exists.** The ePrint note introduces evolving nullifiers only as a sketch: “As a concrete example, assume the nullifier for preventing the double-spend of a note is modified from  $\eta \leftarrow \text{PRF}(k_{\text{note}}, \rho)$  to  $\eta_e \leftarrow \text{PRF}(k_{\text{note}}, \rho \| e)$ ” with  $e$  “a monotonic counter known as an epoch identifier” (ePrint 2025/2031, §1.3). The concrete derivation exists only in the design book (`book/src/nullifiers.md @ 26fdc165`): a Goldreich–Goldwasser–Micali tree of depth  $D$  and

arity  $A$  rooted at the per-note master key

$$\text{mk} = \text{Poseidon}_{\text{Tachyon-NfPrefix}}(\psi, \text{nk}),$$

descended one Poseidon call per base- $A$  chunk of the epoch index, with the epoch- $e$  nullifier one further call under a separate domain,  $\text{nf} = \text{Poseidon}_{\text{Tachyon-NfDerive}}(\text{KDF}_{\psi}^{\text{climb}}(e, D))$ . The note commitment  $\text{cm} = \text{Poseidon}_{\text{Tachyon-CmDerive}}(\text{rcm}, \text{pk}, v, \psi)$  digests  $\psi$  and (through  $\text{pk}$ )  $\text{nk}$ , so one commitment freezes the note’s entire nullifier sequence. Delegation hands a service a window of future nullifiers committed on coefficient generators with a sentinel,  $\delta = \sum_i [\Delta_{e+i}] \mathcal{G}_i + \mathcal{G}_d$ , re-based so any window stands alone. None of this is implemented: the `zcash_tachyon` crate lists “Note commitments” and “Nullifier derivation (GGM tree PRF)” under “Stubbed (pending Ragu PCD and Poseidon integration)” (`README.md` @ `26fdc165`; §4.3), and `crates/tachyon/src/note.rs` calls the commitment scheme “TBD”. The material is single-committer and churning: alternatives remain open in the tracker for the derivation itself (issues #86, #139) and for the absence-proof ranges it feeds (#87, #97, #98; all open, fetched 2026-07-08).

**What a specification must pin down.** The depth  $D$  and arity  $A$  (symbolic in the book; the crate pins working values, §3.4); the Poseidon instances, their parameters, and the byte-exact domain tags; the encoding of the epoch index into chunks; the uniqueness rule for  $\psi$  — the book states that two notes sharing  $\psi$  share  $\text{mk}$  and hence the same nullifier sequence, “so spending one publishes the other’s nullifiers” (`nullifiers.md`), yet no rule assigns responsibility for preventing reuse; the generators  $\mathcal{G}_i$  and the window-commitment format; and the nonzero guards on published nullifiers, which the book motivates only in prose (a zero nullifier “would collide with the note’s own  $\text{cm}$  in the tachygram scan”, `proof-tree.md`).

**What would falsify the design.** A distinguisher on the per-epoch values: obliviousness rests entirely on nullifiers at distinct epochs being unlinkable, so a practical distinguisher for Poseidon-as-PRF in this mode — linking  $\text{nf}_e$  to  $\text{nf}_{e'}$  without  $\text{mk}$  — collapses the construction. Likewise a derivation-chain break: a valid absence proof over values that are not the note’s genuine leaves would let a spender evade the double-spend check the whole system replaces the nullifier set with.

### 5.1.2 The PIR detection design

**What exists.** One sentence. Issue #121 (2026-05-26, verified via the GitHub API 2026-07-08) records the pivot in full: “Since we’re using PIR to make repeated payments flows without OOB, there are stale parts of the book that will need to change. We’re retaining in-band secret distribution for recover from seed flows.” The same issue flags the book’s key-hierarchy chapter as stale, and the chapter carries the marker `<!-- todo: this is significantly outdated -->` (`book/src/keys.md` @ `26fdc165`). Everything else written about payment detection predates this pivot and describes the abandoned fully-out-of-band design (blog “Tachyaction at a Distance”, 2025-05-15): removal of key diversification, viewing keys, and payment addresses, with unlinkability deferred to ZIP-321 payment requests and ZIP-324 encapsulated payments — the latter itself a 2019 draft whose rseed domain separator is the literal `0xFIXME` and which declares the transport channel out of scope (`zips` @ `c6b70358`). The project roadmap describes the PIR-plus-post-quantum-key-exchange payment protocol as developed independently in collaboration with the team; no document of any rigor describes it (`tachyon.z.cash/roadmap`, dateless, fetched 2026-07-08).

**What a specification must pin down.** The PIR family (single-server computational or multi-server information-theoretic, with the corresponding non-collusion assumption); the database the queries run against, who maintains it, and how a detection index is derived from a payment; the query and server cost model as the database grows; the post-quantum key exchange (named

nowhere); and the retained in-band channel — its ciphertext format, its interaction with the removal of viewing keys, and which flows are entitled to it. The design currently consists of a sentence naming a primitive; every one of these is unwritten.

**What would falsify the design.** Cost: single-server PIR touches the whole database per query, so a database that grows with chain history can make detection more expensive than the trial decryption Tachyon set out to eliminate. Privacy: query-pattern or timing leakage at the PIR server re-introduces exactly the correlation channel of §5.2.1. Either failure, demonstrated at realistic scale, refutes the pivot rather than the parameters.

### 5.1.3 The sync-service trust model

**What exists.** Role prose. The ePrint note asserts services are fully oblivious to their clients’ transaction details and keep ephemeral per-client state; the book’s role table is the most precise statement: the sync service runs `UnspentSeed`, `EmptyBlockUnspentSeed`, `UnspentFuse`, and `UnspentEpochFuse` over nullifier values the wallet shared, and “never sees a note, `cm`, `psi`, or `mk`” (`book/src/proof-tree.md` @ 26fdc165). The security requirement is stated by the designers in the negative: “it is essential that the syncing service be incapable of distinguishing between transactions they assisted with and ones they did not” (ePrint 2025/2031, §1.3). The mechanism offered is epoch unlinkability plus proof re-randomization, the latter by citation: the problem “can be generally addressed by re-randomizing the proof, as many recursive proof systems permit, such as those involving accumulation schemes” (ibid., citing Bünz–Chiesa–Mishra–Spooner, ePrint 2020/499). On the implementation side, `ragu` does expose `Application::rerandomize` (§4.2), but its own API documentation warns that fused proof-carrying data is not zero-knowledge until re-randomization is performed, and the re-randomization path rests on an in-code “temporary hack” (`crates/ragu_pcd/src/fuse/mod.rs`, `lib.rs` @ 830bbcda). The corresponding ZIP stub is empty (`book/src/zips/oss.md`).

**What a specification must pin down.** A definition of “oblivious” as a game — what the adversarial service observes (requests, timing, shared windows), what it must not distinguish, and against which collusion (service alone, service plus network observer, several services comparing notes); what state a service may retain across sessions, “ephemeral” being currently a one-word promise; the client–service protocol itself, including handoff between services, which no document describes; and whether re-randomization is mandatory per exchange and on which side. No security definition of any of this exists in the corpus.

**What would falsify the design.** The designers’ own criterion, inverted: a service strategy that distinguishes assisted from non-assisted transactions. The timing-correlation channel of §5.2.1 is the known candidate, and the designers state plainly that its success would be fatal.

### 5.1.4 Epoch parameterization

**What exists.** An open issue. The epoch period — the single parameter the pruning model, the delegation window, the dual-nullifier rule, and the data-availability pressure all hinge on — is untuned: issue #79 (“Tune epoch period”, 2026-05-16, verified via the GitHub API 2026-07-08) records, beyond a pull-request reference, only that it “requires empirical analysis”. The GGM depth  $D$  and arity  $A$ , which bound the number of epochs a note can ever traverse, are symbolic throughout the book; the crate pins working values (§3.4) that no design document yet ratifies. The ePrint note supplies only the qualitative trade: with long epochs (“e.g., years”) the ecosystem “can perhaps organically support” storage of past nullifiers, but payment-scale throughput requires epochs “perhaps even as short as a single block”, which is precisely the regime that concentrates history into few providers (ePrint 2025/2031, §1.4).

**What a specification must pin down.** The epoch length in blocks and its activation semantics;  $D$  and  $A$ , and therefore the lifetime ceiling  $A^D$  epochs a note can survive; the default and maximum delegation window; and the reorganisation behaviour of the two-epoch consensus scan (§5.1.5), which the book does not treat at all.

**What would falsify the design.** The absence of a satisfying point: if every epoch length either (short) grows the historical nullifier database past what unincentivised parties store, or (long) grows the per-note absence-proof work and time-to-spend past what wallets and services sustain, then the parameter the design assumes exists does not. The designers’ own scaling narrative places the target regime at the short end, where their own data-availability warning applies (§5.2.2).

### 5.1.5 The rollover soundness model

**What exists.** A book-only consensus rule and a prose soundness argument. The published design rejects note expiry — the ePrint note considers expiring notes and nullifiers and discards it because “users must move their funds into a new note, once per window, or lose their funds” (ePrint 2025/2031, §1.1) — so any expiry-based description of Tachyon is obsolete; notes stay spendable indefinitely via delegated absence proofs. What replaces expiry is rollover: a spend publishes *two* nullifiers, for the anchor epoch and the next, and consensus checks every tachygram for duplicates over exactly the current and immediately preceding epoch, rejecting a duplicate within that two-epoch window (`book/src/tachygrams.md @ 26fdc165`). Everything older may be discarded: “Validators must only maintain the leading edge of revealed nullifiers (to prevent duplicates in concurrent transactions) but can otherwise permanently prune” (forum t/50789, post #22, 2025-11-04). The ePrint note carries only the looser “validators must still check for duplicates”. The book’s Soundness section argues the composition in prose: splice identities (Construction 3.5) checked at a Fiat–Shamir challenge, commit-equality binding of witnessed sequences, and cm-threading through the spendable lineage (`book/src/proof-tree.md`). There are no formal definitions, no theorems, and no machine-checked statements; the founding post itself concedes that recursive-soundness bounds of PCD constructions are “largely ignored in practice” (blog, 2025-04-02). The tracker carries live soundness and malleability issues against the design: “Bundle commitment is order-free over actions, so bundles are malleable to action reordering” (#137), unchecked padding in proof encodings (#161), and sub-block state landing in stamp order rather than transaction order (#135; all open, fetched 2026-07-08). Even the rule’s modality is unsettled: the anchor chapter hedges, in an in-file comment, between “may”, “should”, and “must” for which pool states consensus acknowledges (`book/src/anchor.md`).

**What a specification must pin down.** The consensus rule as normative text: the exact scan set, the epoch-transition tolerance, whether pruning is a permission or an obligation, and behaviour under reorganisation across an epoch boundary; the tachygram-set commitment format the scan runs over; and a soundness statement with a proof — at minimum, that two spends of one note accepted in any pair of epochs necessarily collide inside some single two-epoch scan window, given the dual-nullifier publication rule, and that the statement survives composition with the recursive proof system’s actual extraction bound.

**What would falsify the design.** A double-spend across the pruning boundary: two accepted spends of one note whose published nullifier pairs never meet inside one scan window. The two-epoch window is the entire replacement for the permanent nullifier set; a counterexample — or a demonstrated action-reordering exploit graduating from the open malleability issues — refutes the state-pruning claim that motivates the project.

## 5.2 Open problems

Each entry below is acknowledged as unsolved by the designers; we quote the acknowledgement. These are not gaps a specification could close by transcription — in each case the missing object is the construction itself.

### 5.2.1 Timing-correlation linkage

**What exists.** The problem statement, in the designers’ words: wallets hold multiple notes and update their unspent proofs together — “such as when first turned on after a period of inactivity, or charging on WiFi at home. Transactions spending these notes to different parties could still be linked together by their use of incremental unspent proofs that were computed at correlated times. This is not a small problem; in fact, it would be fatal to the entire approach” (ePrint 2025/2031, §1.3). The remedy on offer is one clause — re-randomizing the proof, by citation to accumulation schemes (ePrint 2020/499) — plus a gesture in the founding post at “network privacy countermeasures like mixnets” and a promised future post, which never appeared (blog index ends 2025-10-16). Ragu’s `rerandomize` exists as code (§5.1.3), but no published argument connects it to the property the ePrint needs: that a re-randomized proof is indistinguishable from one the service never touched, under the timing side information the service actually has. Collusion of a service with a network observer has no published analysis anywhere in the corpus.

**What a resolution must establish.** A formal indistinguishability property for re-randomized PCD in ragu’s accumulation regime, a proof that `rerandomize` achieves it, and — separately, because re-randomization does not touch timing — a traffic discipline (batching, cover queries, or a mixnet layer with a named threat model) that removes the correlation channel the designers describe. The transport side is currently one word in one blog post.

**What would falsify the design.** The designers have already stated it: a working correlation attack that links two spends through the service that helped maintain both “would be fatal to the entire approach”. This is the only entry in the corpus where the falsification criterion is supplied verbatim by the designers, and it remains open.

### 5.2.2 Data availability, liveness, and incentives

**What exists.** A designer-acknowledged hazard with no mechanism. Validators prune what wallets and services later need to prove absence over (§5.1.5); the history must therefore live somewhere off-consensus. The ePrint note describes the end state at scale: “one could easily imagine a point where the historical nullifier database grows so large that it can only be stored by a few highly centralized providers ... the sudden shutdown of these services could render the funds of all their dependent users unspendable, potentially permanently” (ePrint 2025/2031, §1.4). No new layer is planned: “Tachyon will not involve a new DA layer; the blockchain will continue to be used to post nullifiers and note commitments, but not much else involving Tachyon” (forum t/50789, post #25, 2025-11-16); a DA layer “would probably exist solely to incentivize replication of the historic nullifiers/commitments, and more importantly to incentivize running oblivious syncing services” (ibid.). The designer’s stated position is deferral: “I’m looking forward to talking about this more in the future, but it’s not a concern of mine right now” (post #22, 2025-11-04). The community objection is on the record and unrebutted: building the non-inclusion proof “requires knowledge of every used nullifier” (post #19, 2025-10-29) — someone must keep everything consensus discards. No incentive design of any kind exists.

**What a resolution must establish.** Who stores the pruned nullifier and commitment history, under what incentive, with what replication factor; a liveness bound — how long a wallet’s funds remain spendable after its services vanish; and a recovery path for a user whose absence-proof



lineage lapses over a pruned interval. The ePrint’s own remedy is a sketch of a few paragraphs, framed as a possibility (“The second possibility is to build a form of data-availability (DA) layer”, §1.4), not a design.

**What would falsify the design.** The designers’ scenario realised: the throughput target forces short epochs (§5.1.4), short epochs force a large historical database, storage centralises, and a provider shutdown strands funds — “unspendable, potentially permanently”. A scaling design whose failure mode at its own target scale is permanent loss of user funds, with no incentive mechanism even sketched, is refuted by the first such event; the open problem is to make the event impossible rather than unlikely.

### 5.2.3 Recoverability

**What exists.** An acknowledged regression and a one-sentence mitigation. Removing in-band secret distribution removes the property that the chain itself is the backup: “the user cannot rely on the blockchain as a storage mechanism for recovering their funds from a seed phrase” (blog, 2025-04-02). The designers accept the cost as forced: “Getting this right will be challenging, but we simply have no choice but to do it” (forum t/50789, post #22, 2025-11-04). Asked directly, the answer remains non-committal: the replacement infrastructure “opens up a can of worms like who runs it? ... What if it’s the people building it like don’t exist anymore? How do I get my funds?” (ZK Podcast episode 388 transcript, 2026-01-21). The entire written resolution as of 2026-07-08 is the second sentence of issue #121: “We’re retaining in-band secret distribution for recover from seed flows” — an issue whose first sentence declares the book stale against it. The roadmap’s alternative delivery path warns of “backup/recoverability trade-offs”; a direct forum question about seed-phrase recovery (2025-10-15) went unanswered.

**What a resolution must establish.** A complete recover-from-seed procedure: what the retained in-band channel carries and where it lives in a protocol whose validators prune; how a restored wallet discovers its notes’ creation epochs and positions; and how it rebuilds an absence lineage over history that consensus discarded — which makes recoverability a dependent of the data-availability problem (§5.2.2), a dependency no document states.

**What would falsify the design.** A seed-only restoration that cannot terminate: a user with a correct seed phrase whose funds are unrecoverable because the services holding the required history no longer exist. For a system marketed as a payment protocol upgrade, demonstrated non-recoverability at any realistic scale refutes the wallet model, not a parameter of it.

### 5.2.4 Tip-nullifier exposure versus the obliviousness claim

**What exists.** A marketing claim and a design detail in tension. The project overview states: “the service never learns your actual nullifiers, because the protocol forces them to periodically evolve in an unlinkable way” (tachyon.z.cash/overview, fetched 2026-07-08). The design book’s delegation construction hands the service one nullifier per epoch-boundary crossing “*plus the nullifier of the epoch in progress at the span’s tip, which is carried separately because that epoch is not yet complete*” (book/src/nullifiers.md @ 26fdc165). The tip value is not a spent, superseded nullifier: the spendable’s current nullifier is “the nullifier the wallet would publish to spend now” (ibid.), so a service holding it can watch the pool and recognise the client’s spend in that epoch — linking a service session to an on-chain transaction, exactly the linkage obliviousness is sold as preventing. The role table shows the escape: `UnspentSeed` and the other segment steps are marked *possible* for the wallet (book/src/proof-tree.md), so a wallet can keep the leading edge private and delegate only completed epochs. No document mandates that flow, and none analyses the leak when it is not followed.

**What a resolution must establish.** Either a mandated flow in which the tip nullifier never leaves the wallet — with the wallet-side cost of sealing each epoch stated — or a bound on what a service holding the tip learns, under the timing side information of §5.2.1. Until one of these exists, the overview’s “never” is falsified by the design book’s own construction, and the accurate statement is: the service never learns *past* nullifiers, and learns the current one unless the wallet does extra work no document requires of it.

**What would falsify the design.** Service-side spend detection in the delegated flow — trivially demonstrable today from the book’s data flow. The narrow claim is already false as advertised; what remains open is whether the protocol will mandate the wallet-side variant that makes it true.

### 5.3 Quantitative claims without a published basis

The headline figures attached to Tachyon are not project measurements, and a volume at this series’ standard must say so plainly.

- The homepage states that Tachyon “shrinks transactions by two orders of magnitude and removes runaway state growth for validators” (tachyon.z.cash, fetched 2026-07-08). No published benchmark, size specification, or derivation supports the factor; it refers to marginal on-chain size after aggregation and miner-side stripping, and no primary document computes it.
- The circulating throughput figures — roughly 2,200 TPS, or 660 TPS at  $\sim 1.5$  kB per transaction, and a  $9.3 \text{ kB} \rightarrow 450 \text{ B}$  size reduction — trace to Messari and CoinDesk analyst reports (the latter commissioned by GenZcash, 2026-06-30), not to the project. They are analyst estimates. Bowe’s posts contain no TPS numbers, and the founding post’s promised PCD-toolkit benchmarks (2025-04-02) have not appeared.
- The only in-project performance numbers are **ragu**’s: verifier time “approximately 100ms” at recursion threshold  $K = 11$  and a break-even against Orchard batch verification at roughly 50 transactions — self-labelled “rough estimates that should be validated with empirical benchmarks” (ragu book, `protocol/analysis.md` @ 830bbcd). A criterion bench harness runs in CI and publishes no numbers.
- The claim that syncing services do only “a similar amount of computational effort (asymptotically)” to today’s validators is prefixed “I believe” in its only source (forum t/50789, post #28, 2025-11-21); no complexity analysis or benchmark exists.
- The founding post’s timeline — “many of the major scalability improvements should be able to hit mainnet within a year” (2025-04-02) — is falsified: as of 2026-07-08 nothing runs on testnet or mainnet, no ZIP is upstreamed, the core ZIP stubs are empty, the crate’s proof layer is stubbed, and **ragu** is explicitly pre-audit. The current roadmap carries no dates.

### 5.4 Revisit triggers

The classifications above are dated 2026-07-08 and decay. Any of the following moves material between bins and obsoletes parts of this section: merge of ZIP-248 (the transaction-format substrate every Tachyon bundle design defers to, open as zcash/zips PR #1156); upstreaming of the project’s four fork amendment drafts (PRs #1–#4, open since 2026-06-02); a filled core ZIP stub (#103–#107); a published PIR payment protocol; and **ragu**’s first published benchmark or audit. The update-ZIP integrations remain undecided in the designers’ own words — “still under discussion and nothing has been decided yet” for the ZIP-221 history-tree leaf and the ZIP-317 fee contribution, “blocked on ZIP-248 reaching editor consensus” for bundle registration (book/src/zips/ @ 26fdc165).



## 6 Security posture and outlook

This closing section states the volume’s synthesis. We compress the preceding chapters into a single conservation law (§6.1), place Tachyon at its one point of contact with the post-quantum programme (§6.2), fix its relation to the surrounding consensus roadmap (§6.3), and end with the volume’s verdict: the concrete artifacts that must exist before any Tachyon claim can be checked rather than believed (§6.4). Ground truth is pinned as throughout the volume: the design book at `tachyon-zcash/tachyon @ 26fdc165` (2026-07-07), Ragu at `tachyon-zcash/ragu @ 830bbcd` (2026-07-05), the ZIP corpus at `zcash/zips @ c6b70358` (2026-07-05); web sources carry individual dates. The material compressed here is constructed in §2.1 (the proof-carrying fold and its nineteen steps), §5.1.1 (nullifier evolution), and §5.1.3 (the sync-service model); Ragu’s code state is §4.2; the open problems are §5.2.

### 6.1 The conservation law

The designers claim the scaling gain costs nothing in the two properties Zcash exists to provide. The eprint promises “continual, permanent pruning of nullifiers by validators”, achieved “without compromising privacy”; its syncing services are *untrusted*, hold only “ephemeral state per client”, and “cannot link their clients to any transactions that ultimately appear on the public ledger” (Bowe–Miers, eprint 2025/2031, abstract; revised 2025-11-03). Nothing in the design asks a user to trust a third party with funds or with integrity — a service that lies is caught by proof verification — and the assumption family, Pallas/Vesta discrete logarithms plus random-oracle heuristics, is the one already deployed in Orchard. In this exact sense soundness and privacy are preserved *in principle*.

The qualification carries weight. Soundness of the proof tree is argued in prose (the book’s Soundness section, `book/src/proof-tree.md @ 26fdc165`; the eprint contains zero theorems and no formal security definitions); the term “oblivious” has no security definition anywhere in the corpus; the hiding of the new Poseidon-based commitments is asserted, not proven; and the founding post itself concedes that the recursive soundness bounds of PCD constructions are “largely ignored in practice” (Bowe, “Tachyon: Scaling Zcash with Oblivious Synchronization”, 2025-04-02). Each of these is *designed but unspecified* at best, and none changes bins in this section.

What the design spends is availability. Pruning does not destroy the history a wallet needs; it relocates the obligation to keep it. Validators drop exactly what wallets later require to prove their notes unspent, and someone must hold it. The designers rule out a protocol-level home for that obligation: “Tachyon will not involve a new DA layer; the blockchain will continue to be used to post nullifiers and note commitments, but not much else involving Tachyon” — while observing that under Tachyon users “become more sensitive to the availability of the history in order to make their funds spendable”, and that a data-availability layer, if one arose, “would probably exist solely to incentivize replication” (forum t/50789 #25, 2025-11-16). The computation relocates the same way: the expectation that syncing services will spend “a similar amount of computational effort (asymptotically) that validators currently do” is a designer’s “I believe” (forum t/50789 #28, 2025-11-21) with no published complexity analysis behind it.

We compress the relocation into a pick-two. Deployed Orchard provides three availability properties simultaneously — and pays for them with the unbounded validator state Tachyon exists to remove:

- (i) *full obliviousness*: no helper party learns anything about the wallet’s notes or transactions;
- (ii) *no liveness dependence*: spendability does not require any particular external party to be running;
- (iii) *chain as complete backup*: a seed phrase plus the public chain recover all funds.

Once validators prune, a wallet retains at most two. Keeping (ii) and (iii) means the chain retains everything a recovering wallet needs — that is today’s protocol, not Tachyon. Keeping (i) and (ii) means the wallet ingests and stores the pruned nullifier history itself, continuously; the chain stops being its backup, exactly as the founding post concedes: “the user cannot rely on the blockchain as a storage mechanism for recovering their funds from a seed phrase” (2025-04-02). Keeping (iii) through services — delegated absence proofs reconstructing spendability from seed — surrenders (ii): “the sudden shutdown of these services could render the funds of all their dependent users unspendable, potentially permanently” (eprint 2025/2031, §1.4). And the retained (i) is itself the design’s most fragile member: correlated proof-update timing linking a wallet’s transactions “is not a small problem; in fact, it would be fatal to the entire approach”, with the remedy — proof re-randomization — supplied by citation rather than construction (eprint 2025/2031, §1.3); this volume classifies it an *open problem*.

The project’s own trajectory moves inside this triangle rather than out of it. The May-2026 pivot retains “in-band secret distribution for recover from seed flows” (tachyon-zcash/tachyon issue #121, 2026-05-26), buying back the note-discovery half of (iii) at the price of on-chain ciphertexts for those flows, while spendability from seed still rests on delegated absence proofs and therefore on (ii); the roadmap’s fallback — dedicated note-delivery infrastructure — concedes “UX or backup/recoverability trade-offs” in as many words (tachyon.z.cash/roadmap, fetched 2026-07-08). Classification of the trilemma itself: the framing is this volume’s; every fact inside it is designer-acknowledged and individually cited; and no Tachyon document states the trade-off in one place. A specification will have to, because the chosen corner determines what a wallet must store — the difference between a seed phrase that recovers funds and one that does not.

## 6.2 The post-quantum intersection

The post-quantum analysis of this series (the *Ironwood Guide*, §“Post-quantum security of the Orchard protocol”) reaches one headline conclusion about deployed Orchard: exactly one quantum threat is retroactive. Note encryption is the canonical harvest-now-decrypt-later target — every  $(\text{epk}, C_{\text{enc}})$  pair recorded on chain today becomes decryptable by a future discrete-log-capable adversary who also holds the recipient’s address, exposing amounts, memos, and the full receiving history, permanently — while every integrity break (Sinsemilla binding, value-commitment binding, RedPallas forgery, Halo 2 soundness) is prospective and mitigable by switching pools off in time. The deployed mitigation programme declines to touch the retroactive item: “The situation with respect to Privacy is unchanged for any pool”; the note encryption of the deployed pools “is subject to ‘Harvest Now, Decrypt Later’ attacks”, with countermeasures only “under consideration” (ZIP 2005, Non-requirements and Privacy discussion, zips @ c6b70358).

Tachyon is the only artifact in the corpus that addresses this liability, and it does so structurally rather than cryptographically: it removes the ciphertext. “We plan to remove in-band secret distribution from Tachyon transactions to fully defend against all quantum attacks on privacy in on-chain transactions. This step is essential, because harvest now, decrypt later threats are perilous for blockchain protocols” (Bowe, “Zcash and Quantum Computers”, 2025-10-16). Where no key-agreement ciphertext is published, there is nothing to harvest, and the remaining on-chain objects — hiding commitments, zero-knowledge proofs, re-randomized keys — leak nothing to a quantum adversary who lacks the address. Three qualifications bound the claim. First, the defence is prospective only: ciphertexts already recorded remain harvestable forever; no protocol change can repair them. Second, the pivot of issue #121 retains in-band distribution for recover-from-seed flows, and whether those ciphertexts use the deployed discrete-log key agreement or the “post-quantum key exchange” the roadmap names for its PIR payment protocol is pinned down nowhere — the PIR protocol has no published specification at all (tachyon.z.cash/roadmap, fetched 2026-07-08). Third, no post-quantum privacy analysis of Tachyon exists; the statistical hiding of its Poseidon-based commitments is asserted, not proven. Bin: *designed but unspecified*, and the recovery-flow cryptography is the first thing a specification must fix before the

post-quantum privacy claim becomes evaluable.

On the soundness side the same project deepens the discrete-logarithm commitment. Ragu is an ECDLP-based recursive SNARK by declared design: the requirement of small recursive proofs “essentially limits us to ECDLP-based constructions with our existing payment protocol architecture”, with a footnote recording “reduced concern (in the medium term) for post-quantum *soundness* risks” that cites the same blog post (`book/src/protocol/index.md`, ragu @ 830bbcda). The designers’ asymmetry argument is explicit: “Quantum privacy breaks are usually retroactive, whereas soundness breaks are usually not” (Bowe, 2025-10-16); a prospective soundness failure is answerable by reaction — turnstiles bound inflation, and ZIP 2005 re-anchors note ownership in symmetric primitives so that funds survive a migration. The argument is coherent, and it prices one thing incompletely: depth. Under Tachyon the discrete-log assumption no longer underwrites a single Action circuit but the recursion substrate itself, and the wallet’s state — the proof-carrying data that replaces the chain as the user’s record — is a stack of DL-sound proofs whose recursive soundness bounds the founding post already concedes are “largely ignored in practice” (2025-04-02). A future migration off the assumption is then a migration of the state model, not the replacement of one circuit. Bin: *open problem*, jointly with the unquantified recursion-soundness bound; no document in the corpus analyses either.

### 6.3 Relation to Crosslink and the upgrade environment

Crosslink — the finality/proof-of-stake effort led by Shielded Labs — and Tachyon are formally independent. The founding post sees “no reason to believe that Tachyon will conflict with any of the active areas of development such as Crosslink and ZSAs” (2025-04-02); Shielded Labs states that “Crosslink and Tachyon are separate efforts that don’t necessarily interact with each other with regards to roadmapping + timelines” (forum t/50789 #11, 2025-10-28); and no interaction analysis exists in either direction — a repository-wide search of the ZIP corpus finds Crosslink only in ZIP 218 (25-second block target spacing, Status: Draft, created 2026-03-13; zips @ c6b70358, searched 2026-07-08): one body sentence declaring the proposal “complementary to, and does not compete with,” finality mechanisms such as Crosslink, plus the citation footnote. The independence is therefore asserted, not examined; both proposals alter the environment the other assumes (consensus finality and block cadence on one side, validator state and epoch-windowed double-spend checking on the other), and Tachyon’s epoch period is itself untuned (tachyon-zcash/tachyon issue #79). The near-term environment is fixed without Tachyon: NU6.2 is settled (ZIP 257, Status: Final — the emergency Action-circuit correction), NU6.3 carries the Ironwood pool and the ZIP 2005 recoverability programme (ZIPs 258, 229, 2005; Draft/Draft/Proposed), and Tachyon’s own standing is a mandate without a vehicle: “universal or near-universal support across every group ... the most unambiguous signal from the entire poll” in the February 2026 NU7 sentiment poll, with the stated next step only to “assess technical readiness and scope for inclusion in NU7” (zfnd.org, NU7 Polling Results, 2026-02-23), while the deployment draft (`draft-arya-deploy-nu7.md`, heights TBD, zips @ c6b70358) lists no Tachyon ZIP. Beyond NU7 there is nothing to relate Tachyon to: the ZIP corpus contains no NU8 artifact of any kind (repository-wide search of zcash/zips @ c6b70358, 2026-07-08); the relation to NU8 becomes statable only when an NU8 deployment draft exists.

### 6.4 The checkability boundary

Every claim in this volume carries one of three labels: *specified*, *designed but unspecified*, or *open problem*. The distribution of those labels is the verdict. The specified bin contains standing and environment facts only — the poll result, editor affiliations, the NU6.2/NU6.3 context, the ZIP 321/324 out-of-band substrate, and Ragu’s code state — and not one byte format, consensus rule, or security theorem of Tachyon itself. Three artifact classes would move the boundary.

**Benchmarks.** No performance figure exists at project rigor. The founding post promised benchmarks of a proof-carrying-data toolkit “to set a floor on the performance of shielded transaction aggregation and oblivious syncing services” (2025-04-02); none has appeared. The only in-project numbers — roughly 100 ms aggregated verification at recursion threshold  $K = 11$  and a break-even against Orchard batch verification near 50 transactions — are self-labelled “rough estimates that should be validated with empirical benchmarks” (`book/src/protocol/analysis.md`, ragu @ 830bbcd); the CI bench harness publishes no output; the circulating throughput and size figures are analyst estimates, not project figures; and the homepage’s two-orders-of-magnitude headline has no published derivation. A meaningful benchmark moreover requires code that does not yet exist: Ragu’s `Proof` carries full polynomials, verification is linear-time, and no inner-product opening argument or decider is implemented (`crates/ragu_pcd/src/{proof/mod.rs,verify.rs}` @ 830bbcd). Until a benchmark of non-mock Ragu PCD with succinct proofs exists, every scaling claim is unfalsifiable.

**The nullifier-evolution specification.** The one genuinely novel cryptographic object — the evolving nullifier — exists at eprint-sketch and design-book rigor only: the eprint offers a PRF sketch “as a concrete example” (2025/2031, §1.3); the concrete GGM/Poseidon derivation, the dual-nullifier spend, and the two-epoch duplicate window live only in the book (`book/src/{nullifiers,tachygrams}.md` @ 26fdc165); the crate stubs the derivation and calls the commitment scheme “TBD” (`crates/tachyon/src/note.rs`); the derivation is churning through open alternatives (issues #139, #86, #87/#97/#98) with live soundness and malleability issues (#161/#137/#135); and the epoch period, GGM depth, and arity on which the entire pruning model hinges are untuned (issue #79, “requires empirical analysis”). A specification must pin the derivation formulas and domain separators, the epoch parameters, the dual-nullifier consensus rule and the pruning rule, and — above all — the security definitions (per-epoch unlinkability, service obliviousness) that nothing in the corpus states formally.

**Core ZIPs.** The consensus path is empty. The five core ZIPs that would carry the protocol — Shielded Protocol (#103), Bundle (#104), Accumulator (#105), Aggregator (#106), OSS (#107) — are headers-only stubs of 216–245 bytes each (`book/src/zips/` @ 26fdc165); the four peripheral amendment drafts sit unmerged on the project’s own fork (tachyon-zcash/zips PRs #1–#4, opened 2026-06-02, open as of 2026-07-08); upstream zcash/zips carries zero Tachyon ZIPs; and the extensible-format substrate everything defers to, ZIP 248, is itself an unmerged PR idle since 2026-05-26 (zcash/zips#1156). Beside the ZIPs stand Ragu’s own declared gates: a Fiat–Shamir domain tag that is literally `b"FIXME"` (`crates/ragu_pcd/src/lib.rs:50--51`), registry binding flagged `FIXME(security)` as insufficient (`crates/ragu_circuits/src/registry.rs:673--692`), three missing verifier checks (`crates/ragu_pcd/src/verify.rs:122`), and no audit: “Ragu is under heavy development and has not undergone auditing” (`README.md` @ 830bbcd).

The events that would obsolete this volume’s classification are the revisit triggers of §5.4, read forward. Until they occur, the posture admits a one-sentence summary: soundness and privacy are conserved in intention, availability is spent by design, and none of it is yet checkable.